

四川大學

本科生毕业论文(设计)



题 目	基于 eBPF 的开源 AI 推理引擎的跨语言调用链分析		
学 院	网络空间安全学院		
专 业	网络空间安全		
学生姓名			
学 号		年 级	
指导教师			

教务处制表
2026 年 6 月

基于 eBPF 的开源 AI 推理引擎的跨语言调用链分析

专业 网络空间安全

学生

指导教师

摘要: 随着开源大语言模型在企业私有化部署中的广泛普及,以 Ollama 为代表的推理引擎采用 Go-CGO-C++ 三层混合编程与双进程解耦架构,在系统层面引入了前所未有的观测盲区:当恶意攻击者通过 API 发起目录遍历攻击或畸形载荷注入时,安全运维人员既无法追溯外部数据在推理引擎内部的完整流转路径,也无法回答该行为在内核层面触发了哪些系统调用。

本文设计并实现了 HybriChain——一个面向多语言推理引擎的跨语言调用链追踪与安全分析系统。针对跨语言符号映射难题,提出了五层符号分类体系 (f_0 - f_4),将来自不同命名空间 (Go 运行时、CGO ABI、底层计算库) 的函数符号统一映射到同一语义空间;针对追踪开销难题,设计了基于 eBPF 的探针下沉策略,仅对推理计算核心函数挂载 uprobe,将运行时开销降至最低;针对噪声数据难题,设计了五步漏斗式降噪流水线,将原始事件流压缩 2-3 个数量级;针对调用链缝合难题,提出了基于时间戳对齐的置信度缝合算法,为每条跨语言边赋予 Confirmed/Inferred/Unknown 状态;在此基础上,构建了异常行为分析引擎,支持对 futex 阻塞、异常 mmap 分配、敏感路径访问等高危行为的自动化检测。

本文以 Ollama 为典型靶点验证了 HybriChain 的有效性。实验结果表明,该系统能够从原始内核事件流中还原出完整的跨语言调用链,准确识别推理引擎中的异常行为,为开源 AI 推理软件的可观测性与安全审计提供了新的技术手段。

关键词: eBPF; 开源 AI 推理框架; 跨语言程序分析; 调用链追踪; Ollama; HybriChain

eBPF-Based Cross-Language Call Chain Analysis for Open-Source AI Inference Engines

Cyberspace Security

Student: Adviser:

Abstract: Open-source large language models are now widely deployed in enterprise private environments. Inference engines like Ollama use a Go-CGO-C++ tri-layer hybrid architecture with dual-process decoupling. This design creates significant observability blind spots at the system level. When attackers exploit API endpoints for directory traversal or inject malformed payloads, security operators have no way to trace how external data flows through the inference engine internally, nor can they identify which kernel-level system calls the behavior triggers.

This thesis presents HybriChain—a cross-language call chain tracing and security analysis system built for multi-language inference engines. The cross-language symbol mapping problem is addressed through a five-layer symbol classification system (f_0 – f_4), which unifies function symbols from different namespaces—Go runtime, CGO ABI, and low-level computing libraries—into a shared semantic space. Tracing overhead is minimized by an eBPF-based probe sinking strategy that attaches uprobes only to core inference functions. Noisy data is handled by a five-step funnel noise reduction pipeline compressing raw event streams by 2–3 orders of magnitude. For call chain stitching, a timestamp-aligned confidence algorithm assigns Confirmed, Inferred, or Unknown states to each cross-language edge. On top of these components, an anomaly behavior analysis engine detects high-risk behaviors such as futex blocking, abnormal mmap allocation, and sensitive path access.

Experiments using Ollama as the target validate the system. HybriChain reconstructs complete cross-language call chains from raw kernel event streams and identifies anomalous behaviors in inference engines. The results suggest a practical approach for observability and security auditing of open-source AI inference software.

Key Words: eBPF; open-source AI inference framework; cross-language program analysis; call chain tracing; Ollama; HybriChain

目录

1	绪论	1
1.1	研究背景与意义	1
1.2	国内外研究现状	2
1.3	论文主要工作	2
1.4	论文组织与结构	3
2	相关技术综述	4
2.1	大语言模型推理架构的硬件极限与系统级突围	4
2.1.1	底层硬件制约与架构分析	4
2.1.2	应对内存危机与边缘部署方案	4
2.1.3	AI 基础设施的安全脆弱性暴露	5
2.2	eBPF：重塑 Linux 内核与微服务可观测性范式	5
2.2.1	eBPF 基础架构与核心数据传输	5
2.2.2	将 eBPF 推向应用层服务质量与 AI 黑盒	6
2.2.3	内核态观测所带来的系统级安全对抗	6
2.3	跨语言系统边界的消解与全栈漏洞挖掘分析	7
2.3.1	Go-CGO 互操作与混合图谱重建策略	7
2.3.2	追踪多语种静态数据流（污点分析）的进阶演化	7
2.4	动态二进制插桩体系的底层安全基石作用	8
2.4.1	DBI 的轻量级透明性与架构统一	8
2.4.2	内存级别极端监控的“影子”机制确立	8
2.5	本章小结	8
3	HybriChain 系统设计	10
3.1	问题背景	10
3.2	设计目标	10
3.3	系统整体架构	10
3.4	函数调用追踪方案	11
3.4.1	追踪哪些东西	11
3.4.2	探针放置位置	12
3.4.3	eBPF 的优势	12
3.5	数据降噪与事件解析	13
3.6	调用链缝合	14
3.7	跨语言符号分类体系	15
3.8	异常检测规则	16
3.9	本章小结	16
4	HybriChain 实现方案	17
4.1	Ollama 追踪的特殊问题	17

4.2	eBPF 追踪脚本的具体实现.....	17
4.2.1	uprobe 挂载的具体函数.....	17
4.2.2	系统调用 tracepoint 的具体配置.....	18
4.2.3	Go 函数不可挂 uretprobe 的处理方式.....	19
4.3	数据解析脚本的具体实现.....	19
4.4	静态符号提取的具体实现.....	19
4.5	缝合模块的具体实现.....	20
4.6	异常检测规则的具体实现.....	22
4.7	完整追踪流程.....	23
4.8	本章小结.....	24
5	实验设计与结果分析.....	25
5.1	实验环境.....	25
5.2	调用链缝合准确性验证.....	25
5.2.1	Ground Truth 基准数据集.....	25
5.2.2	缝合评估结果.....	25
5.2.3	缝合结果 Listing.....	26
5.3	eBPF 追踪对推理性能的影响评估.....	26
5.3.1	推理时延 (TTFT) 影响分析.....	26
5.3.2	内存占用分析.....	27
5.4	数据清洗流水线效率分析.....	28
5.4.1	事件压缩效果.....	28
5.4.2	分析处理速度.....	28
5.4.3	过滤正确性验证.....	29
5.5	系统调用分布与推理阶段分析.....	30
5.5.1	系统调用分布特征.....	30
5.5.2	推理阶段划分.....	30
5.6	异常行为发现与案例分析.....	31
5.6.1	异常告警检测结果.....	31
5.6.2	典型案例分析.....	32
5.7	HybriChain 综合评估.....	32
5.8	本章小结.....	33
6	总结与展望.....	34
6.1	论文工作总结.....	34
6.2	研究局限与不足.....	34
6.3	未来工作展望.....	35
6.4	本章小结.....	35
	参考文献.....	36

声 明	38
学位论文使用授权书	38
四川大学本科毕业论文 AI 工具使用声明	39
致 谢	40

1 绪论

1.1 研究背景与意义

近年来，以大语言模型（Large Language Model, LLM）为代表的开源人工智能软件迎来了爆发式的增长，彻底重塑了计算基础设施的演进方向。为了满足数据主权、商业机密保护与隐私合规的苛刻要求，在边缘计算设备和企业内部私有化部署大语言模型逐渐成为主流。在此背景下，开源社区涌现出了如基于纯 C/C++ 实现的 llama.cpp^[1] 及其底层张量代数库 ggml^[2] 等极其优秀的轻量级推理引擎。

然而，大语言模型的参数规模呈现出指数级的膨胀，将底层硬件的内存带宽与计算能力的博弈推向了极端的白热化阶段。X. Ma 与计算机体系结构泰斗 D. Patterson 在其合作的综述中，明确将这种由大规模权重加载与解码（Decode）阶段自回归特性所引发的“内存受限（Memory Bound）”定义为现代 AI 硬件面临的严峻“推理危机”^[3]。A. M. Jain 等人的实证研究进一步表明，在生产级聊天机器人架构中，随着并发请求的扩展，系统尾部延迟急剧上升，硬件算力的物理极限已被触及^[4]。为了应对这种危机，底层系统软件层面开始主导架构重构。W. Kwon 等人提出了教科书级别的 PagedAttention 机制，将现代操作系统的虚拟内存与分页管理思想引入 AI 领域，彻底改变了键值缓存（KV Cache）的内存分配逻辑^[5]。这种由软硬件协同带来的极致性能优化，不可避免地导致了现代 AI 推理框架在系统架构上极度复杂，普遍采用多进程解耦与多语言混合编程范式（如 Go、Python 与 C/C++ 混编）。

这种极其复杂的软件工程实践，使得推理引擎内部的数据流转行为对网络空间安全运维人员而言几乎完全不透明。随着这些多语言框架在生产环境中的广泛应用，其针对数据传输应用编程接口（API）的安全脆弱性逐渐暴露。例如，在 PyTorch 官方支持的 TorchServe 模型部署工具中，由于其架构允许未经严格身份验证的远程用户通过 API 上传模型存档，导致了在反序列化阶段触发极其严重的远程代码执行（RCE）漏洞（CVE-2023-43654）^[6]。类似地，安全研究机构 Wiz Research 披露了 Ollama 框架的 CVE-2024-37032（Problama）高危漏洞^[7]。该漏洞的底层成因是，Ollama 守护进程在处理高层 API 传输的模型数据摘要（Digest）时，未能严格验证其十六进制格式规范，攻击者通过构造携带路径穿越符的畸形 API 数据，成功绕过系统沙箱，实现了对宿主机底层任意核心文件的越权读取。

上述真实的工业界安全事件暴露了一个根本性的技术断层：在多语言混合编程架构下，从顶层应用 API 接收数据负载，到中层编程语言（如 Go 到 C++）跨界桥接序列化，再到底层操作系统内核调用的完整链路，在现有可观测工具下无法被连贯还原。当发生恶意入侵时，系统无法回答“外部注入的数据最终触碰了哪些内核物理内存”。因此，开展针对开源 AI 软件的数据传输 API 调用链分析，实现从高层语义到低层内核执行的完整追踪与异常行为审计，对提升下一代开源大模型应用底座的安全性、稳定性与可观测性，具有极其迫切的理论价值与工程指导意义。

1.2 国内外研究现状

在应对庞杂系统的可观测性与安全审计问题上，学术界与工业界在动态内核追踪、静态跨语言程序分析以及动态二进制插桩等领域展开了大量探索，但针对多语言大模型架构的数据流分析依然存在显著的方法论缺失。

在内核动态追踪与系统可观测性领域，扩展的伯克利数据包过滤器（eBPF）经历了革命性的演进。自 A. Starovoitov 提交核心补丁将其重塑为通用内核虚拟机以来^[8]，B. Gregg^[9] 与 D. Calavera^[10] 奠定了其在系统性能剖析领域的主导地位。随着 A. Nakryiko 引入 BPF Ring Buffer 解决并发数据无损回传^[11]，以及 Cilium 团队开源纯 Go 语言的 eBPF 交互库^[12]，其应用边界不断拓宽。Y. Hu^[13]、M. Ibnath^[14] 以及 B. Zou^[15] 等研究者相继将 eBPF 应用于无服务器（Serverless）网络监控、微服务流式延迟测量以及大语言模型底层的算子级硬件剖析。在安全防护层面，ControlPlane^[16]、LinuxSecurity^[17] 与 Groundcover 团队^[18] 对 eBPF 的威胁模型及替代传统 Auditd 的微隔离实践进行了深度论述。

在跨语言边界的程序分析方面，由于语言环境的割裂，控制流极易断裂。针对 Go 语言调用 C/C++ 的互操作机制，Go 官方团队的 CGO 工具链^[19] 与 J. Chen 等人的实证研究^[20] 揭示了其在类型安全上的巨大隐患。M. Hu 等人^[21]、中科大安全实验室的 POLYCALL 项目^[22] 以及 R. Sunil^[23] 分别尝试通过图转换算法、WebAssembly 中间表示以及大语言模型智能推断来重建多语言混合图谱。同时，在跨语言数据流（污点分析）领域，Brucker^[24]、CHARON 团队^[25] 以及 X. Cheng 提出的 Deepwukong 图神经网络漏洞检测模型^[26]，展示了在静态环境下溯源跨界污染流的巨大潜力。

在作为系统防御终极基石的动态二进制插桩（DBI）体系中，D. Bruening 的 DynamoRIO^[27]、C. Lattner 的 LLVM 架构^[28] 以及 N. Nethercote 开发的 Valgrind^[29] 及其影子内存核心机制 Memcheck^[30]，确立了程序运行时极端监控的理论体系。然而，大模型推理场景在算力与内存吞吐上的极致要求，使得上述重量级的分析方法面临着严重的落地困境。

1.3 论文主要工作

本文聚焦于开源 AI 软件的数据传输 API 调用链分析这一前沿且复杂的系统级难题，针对混合编程架构带来的控制流断裂与观测盲区，设计并实现了一套动静结合、兼顾极低开销与高语义保真度的底层追踪与安全审计框架。论文的主要创新与研究工作包括：

第一，构建跨语言数据传输符号异构映射机制。针对 API 载荷在 Go 语言 CGO 编译期遭遇的符号哈希加扰与 C++ 端复杂的名字重整（Name Mangling）导致的控制流断崖，设计多级抽象符号映射算法。通过深度遍历二进制符号表并提取编译器链接元数据，成功建立从高层路由处理 API 到底层计算执行算子的等价映射链路，修复了多语言程序的静态拓扑。

第二，实施轻量级 eBPF 探针深度下沉战略。摒弃传统的进程级拦截思路，将 eBPF 探针精准挂载于大语言模型推理进程的数据流转咽喉节点。实现了对 API 跨进程通信（IPC）载荷序列化、底层大规模张量 mmap 内存映射调度以及并发线程 futex 锁机制的纳秒级极

低开销捕获。

第三，提出动静结合的 API 调用链融合缝合算法。以纳秒级高精度系统时间戳与线程上下文 (PID/TID) 为纽带，将提取的静态跨界符号图谱与动态 eBPF 环形缓冲区上报的事件轨迹进行高维状态融合。通过基于滑动时间窗口的匹配模型，自动补全丢失的跳转边，生成端到端具备强因果逻辑的数据传输 API 调用链。

第四，开发基于底层调用链的异常安全分析引擎。基于缝合后的高质量完整数据流转轨迹，提炼大模型推理场景下的核心威胁模式。设计多维度规则过滤矩阵，实现了对类似 Problama 的目录遍历越权访问、畸形载荷导致的级联内存锁死等高危异常 API 行为的自动化阻断与研判。

1.4 论文组织与结构

本文共分为六个章节，各章内容安排如下：

第 1 章为绪论，阐述大语言模型推理引擎的硬件极限挑战与多语言混合编程带来的安全可观测性断层，明确本文的研究背景与意义，并对国内外相关研究现状进行综述。

第 2 章为相关技术综述，系统剖析主流推理引擎的多语言混合架构、eBPF 内核可观测性技术、跨语言程序静态分析方法以及动态二进制插桩体系，评述现有技术在应对多语言 AI 安全审计时的局限。

第 3 章为 HybriChain 系统设计，从问题定义出发，阐述“为了 xxx 所以做 xxx”的论证主线——为了获得跨语言调用链，首先实现高效且完备的函数调用追踪；为了让原始数据可用，进行高效统一的调用链合成与降噪；为了让缝合结果可解释，建立准确的跨语言符号映射；最后在上述成果基础上设计异常检测规则引擎。

第 4 章为 HybriChain 实现方案，以 Ollama 推理引擎为典型靶点，与第三章的设计方案形成一一对应的闭环，详细阐述各模块的具体实现与关键算法落地。

第 5 章为实验评估，在多并发高负载的大模型场景下，对 HybriChain 的调用链还原准确率、置信度分布、追踪性能开销以及异常检测能力进行定量验证。

第 6 章为总结与展望，对全文工作进行总结，梳理本文的主要贡献与局限性，并对下一代系统可观测性技术进行展望。

2 相关技术综述

为了实现对开源大语言模型复杂系统内部的深度穿透，准确实施针对数据传输 API 的调用链分析，本章将结合现代计算基础设施前沿技术的演进轨迹，对涵盖大模型推理硬件限制、eBPF 内核可观测性、跨语言程序图谱构建以及动态二进制底层追踪等领域的核心学术文献与工业界标杆进行穷尽式剖析。特别强调每一项关键研究在其技术节点上究竟解决了哪些核心挑战，以及在应对多语言 AI 安全审计时暴露出何种未决的局限性。

2.1 大语言模型推理架构的硬件极限与系统级突围

现代大模型的迭代极大地放大了计算架构中的根本性矛盾，迫使开发者在系统软件层进行极其复杂的工程优化，这些优化构成了跨语言调用链追踪的客观背景。

2.1.1 底层硬件制约与架构分析

X. Ma 与 D. Patterson^[3] 详细剖析了大型语言模型推理硬件的发展趋势与物理局限，他们通过对高带宽内存（HBM）堆叠成本与算力之间博弈的量化研究，解决了学术界对于“内存墙（Memory Wall）”制约大模型算力发展的宏观认知问题，明确指出了单纯堆砌硅片的路径不可持续。但其研究的局限在于，该分析高度聚焦于纯芯片与硬件互联拓扑的物理特性探讨，未能给出应用层系统软件突破这些瓶颈的具体代码级干预方案。

A. M. Jain 等人^[4] 通过构建合成负载生成器与统计学模型评估架构扩展性，解决了预测生产级聊天机器人在高并发长文本输入下发生尾部延迟（Tail Latency）爆炸的定量难题。但还有待解决的问题是，其研究将 AI 框架本身视为黑盒系统，仅在宏观负载层面测量输出，无法解释延迟激增在系统内部引发的跨层级拥塞机理。

2.1.2 应对内存危机与边缘部署方案

W. Kwon 等人^[5] 提出了著名的 PagedAttention 机制，他们利用操作系统经典的虚拟内存按需分页（On-demand Allocation）哲学，彻底解决了 LLM 推理时海量 KV Cache 必须连续分配导致的外部物理内存碎片化问题，实现了吞吐量的几何级攀升。然而不足之处在于，该机制极度优化了内存复用率，却缺乏对跨并发请求在混合内存块池中相互越界读写的深入安全隔离探讨。

G. Gerganov^[1,2] 开发了具有划时代意义的纯 C/C++ 推理库 llama.cpp 及其核心张量库 ggml。他们通过实现深度的整数级混合量化（Quantization）技术与全平台指令集优化，解决了在缺乏专用 GPU 的边缘设备与消费级处理器上部署千亿级参数大模型的难题。但该框架面临的工程挑战是，由于追求极致底层的纯 C/C++ 控制，导致其难以直接且安全地对接现代基于 Go 或 Python 的高并发网络微服务 API 层，从而衍生出了错综复杂的跨语言封装层。

图 2-1 展示了这一架构的典型形态：Ollama 在 Go HTTP Handler 层接收 API 请求后，经由 CGO 绑定层自动生成的桥接函数，最终调用底层 llama.cpp C++ 层完成推理计算。这种跨越 Go → CGO → C++ 三层语言的调用链，正是 HybriChain 需要追踪的核心对象。

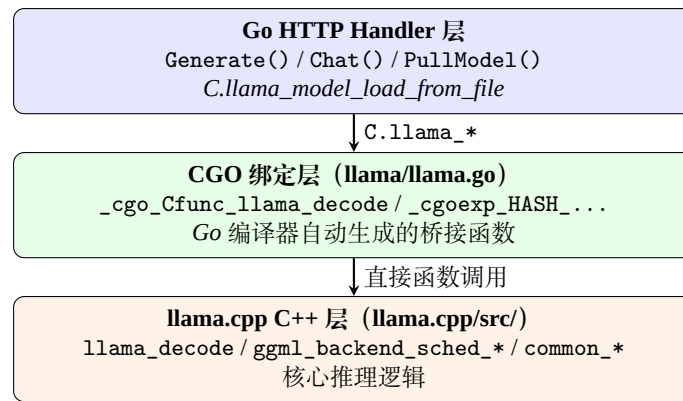


图 2-1 Ollama 混合编程架构示意

2.1.3 AI 基础设施的安全脆弱性暴露

PyTorch 团队维护的 TorchServe 项目^[6]作为标杆性的部署工具，解决了深度学习模型提供 REST/gRPC API 与多模型并发动态批处理的业务痛点。但其在演进中暴露出的问题是，由于追求接口的绝对灵活，早期架构在反序列化阶段允许动态模型注册，导致遭遇了类似 CVE-2023-43654 这样通过构造恶意 YAML 存档进行远程代码执行（RCE）的毁灭性漏洞。

S. Tzadik 领导的 Wiz 安全团队^[7]详细披露了 Ollama 框架的 CVE-2024-37032 高危漏洞（Problama）。他们分析了攻击者如何利用该系统底层的解析缺陷，解决了业界对该框架在获取模型 API 数据包摘要（Digest）时由于未严格验证十六进制格式规范而引发“目录遍历攻击”的具体机理认知。但现阶段亟需解决的系统性难题是，传统安全修复仅停留在事后的打补丁与特定路径过滤上，完全未能在底层内核层面建立一套自动化、实时溯源此类畸形 API 数据从高层网络插口流动到敏感系统文件读写的监控追踪链路。

2.2 eBPF：重塑 Linux 内核与微服务可观测性范式

为了从根本上穿透大模型框架的双进程与跨语言壁垒，eBPF 技术展现出了独一无二的优越性，其演进史本身就是一部寻求在系统性能与监控深度之间完美平衡的进化史。

2.2.1 eBPF 基础架构与核心数据传输

A. Starovoitov^[8]提交的一系列历史性重构补丁，解决了传统 BPF 只能受限于网络子系统的僵化局面，通过将其升格为通用内核虚拟机并引入极其严格的代码静态验证器（Verifier），赋予了用户态在内核极速且安全执行观测逻辑的能力。基于此架构，B. Gregg^[9]和 D. Calavera^[10]分别解决了传统系统性能剖析与网络层监控无法兼顾纳秒级开销与全栈透视的工程痛点，奠定了可观测性新范式。

为了突破高频追踪数据的回传瓶颈，A. Nakryiko^[11]引入了划时代的 BPF Ring Buffer 数据结构。他通过构建多生产者单消费者（MPSC）共享内存队列，彻底解决了早期 Perf Buffer 在多核高并发大流量下频繁发生的事件拥塞丢失与时间戳乱序（Event Re-ordering）乱象。随后，Cilium 与 Cloudflare 联合团队^[12]开发了 cilium/ebpf，解决了云原生环境中

Go 语言无法高效、原生地下发与解析 eBPF 字节码的障碍。但这些底层能力依然存在局限，即纯粹的数据传输通道自身并不具备对复杂 AI API 负载进行跨语言重组的高级语义感知能力。

表 2-1 从执行位置、性能开销、安全验证、Go 兼容性等维度，将 eBPF 与 strace、GDB、DBI (Pin)、perf 等传统追踪技术进行了横向对比。如表中所示，eBPF 在内核态安全沙箱中运行，由 Verifier 提供运行时安全验证，同时具备极低的性能开销与对 Go 1.17+ 良好的兼容性，这些特性使其成为追踪多语言混合推理引擎的最佳候选技术。

表 2-1 eBPF 与其他追踪技术对比

维度	strace	GDB	DBI (Pin)	eBPF	perf
执行位置	用户态 (ptrace)	用户态	用户态 (运行时注入)	内核态 (安全沙箱)	内核态
性能开销	高	极高	高	低	低
安全验证	无	无	无	有 (verifier)	有
Go 1.17+ 兼容	可用	需调试信息	危险 (ABI 不兼容)	推荐 (无侵入)	可用
学习曲线	低	高	高	中	中

2.2.2 将 eBPF 推向应用层服务质量与 AI 黑盒

Y. Hu^[13] 创新性地 将 eBPF uprobe 探针应用于 WebAssembly (Wasm) 运行时，解决了在无服务器 (Serverless) 沙箱环境中无法侵入源码即可进行精准 HTTP 流量统计的盲区问题。M. Ibnath 等人^[14] 提出的 eBeeMetrics 框架，解决了系统在缺乏离线反馈分析的条件下，跨多种网络协议精准评估微服务吞吐与尾部延迟的难题。但以上宏观观测研究均存在缺陷，即将庞大的进程视为无状态的黑盒中继站，无法深入探究数据变量在跨语言进程内部的流向。

针对大型人工智能模型底层的黑盒痛点，B. Zou 等人^[15] 提出的 ProfInfer 框架实现了史无前例的降维打击。他们利用 uprobe 结合硬件性能监控计数器 (PMC)，解决了在 Token 生成解码粒度上，精确追踪底层张量算子 (如矩阵乘法) 导致 L3 缓存未命中 (Cache Refill) 等微观硬件物理指标的难题。然而令人遗憾的是，该系统的高度专一化导致其完全放弃了对顶层数据传输 HTTP API 与底层 C++ 计算之间跨语言反序列化语义的缝合，当面对应用层 API 发起的利用攻击时，该框架无法提供任何溯源保护。

2.2.3 内核态观测所带来的系统级安全对抗

ControlPlane 的系统白皮书^[16] 与 LinuxSecurity 的深度对比分析^[17]，解决了业界在选择传统日志记录器 (如 Auditd) 与 eBPF 微隔离方案时的理论分歧。他们指出 Auditd 的事后诸葛亮机制无法抵御现代级联攻击，而 eBPF 在指令执行前 (Pre-execution) 拥有微秒级阻断优势。同时，Groundcover 团队^[18] 解决了 eBPF 在复杂 Kubernetes 容器网格环境中的高可用部署最佳实践问题。但安全社区依然面临挑战：由于 eBPF 的绝对权力，如何防范恶意构造的 BPF 程序进行拒绝服务攻击或越权读取，必须引入极其克制的最小特权 (Least Privilege) 模型，而当前针对 AI 系统内部特定 API 行为定制的轻量级细粒度隔离策略尚属空白。

2.3 跨语言系统边界的消解与全栈漏洞挖掘分析

“多语种编程”是当代复杂架构的常态，但不匹配的类型系统与异构编译器，直接导致了传统单语言静态分析的集体失效，学术界对此展开了猛烈的技术攻坚。

2.3.1 Go-CGO 互操作与混合图谱重建策略

Go 官方团队长期维护的 CGO 核心工具链^[19]，解决了 Go 这门具有自动垃圾回收 (GC) 特性的云原生语言与系统级 C 代码进行底层交互的互操作基础问题。针对该机制的工程滥用，J. Chen 等人^[20]开展了海量的开源项目实证扫描。他们总结出多达 19 种特定于 CGO 的致命缺陷模式，彻底解决了学术界对 Go 语言中滥用互操作会导致类型安全遭到破坏、指针逃逸以及引发严重内存崩溃的感性认知，提供了坚实的量化数据。但其研究局限于静态规则扫描阶段，一旦程序编译打包，静态工具便无法在运行时的复杂并发环境中拦截真实的越界访问。

为了跨越控制流断层，M. Hu 等人^[21]提出了一种极具创意的图转换 (Graph Transformation) 算法。他们通过提取不同语言接口层的抽象模型并执行节点融合，解决了在诸如 Python 调用动态 C 库 (.so) 时的外部映射断链问题。但该方法的严重局限在于，其依赖于动态链接库中暴露的标准导出符号表；由于大语言模型推理框架追求极致效率，多采用静态链接 (Static Linking) 将 Go 与 C++ 打包，导致该方法完全失效。

在此背景下，中科大安全实验室提出的 POLYCALL 框架^[22]采取了降维打击。他们主张直接将所有高级语言统一下沉编译至 WebAssembly (Wasm) 中间表示，成功解决了因为语法差异巨大而无法进行统一函数索引的难题，挖掘出了数百倍于源码分析的隐藏调用边。但其未能解决的挑战是，静态图谱极其庞大且缺乏时间维度，无法捕获实际高频调用中每个 API 载荷的真实状态分布，且极难处理现代编译器故意强行注入的随机哈希加扰 (Hash Mangling)。此外，R. Sunil^[23]尝试引入大型语言模型本身的强大内化知识库，解决了人工推导跨语言映射规则极其低效的问题。但目前的致命缺陷在于，大语言模型生成的控制流图经常出现严重的“推理幻觉”，其产出的拓扑置信度远未达到网络安全领域所需的苛刻取证标准。

2.3.2 追踪多语种静态数据流（污点分析）的进阶演化

针对跨域数据流动引发的沙箱逃逸，早期 Brucker 团队的研究^[24]解决了揭示混合移动架构 (Hybrid Mobile Apps) 下，网页端 JavaScript 恶意向底层原生组件传输异常数据流带来风险的问题。

随后，CHARON 框架^[25]标志着此类技术真正走向成熟。他们开创性地在内存中动态链接不同语言的代码属性图 (CPG)，解决了高度复杂的数据污染流在高级脚本与底层 C 原生扩展层之间“来回多次反复穿梭”的致命追踪难题，一举协助发现了数百个潜伏多年的跨界缓冲区溢出漏洞。结合 X. Cheng 等人提出的 Deepwukong 系统^[26]，他们利用深度图神经网络模型自主学习漏洞在图结构中的拓扑分布模式，解决了依赖人工编写繁杂静态正则表达式检测未知变种漏洞的瓶颈。然而，无论图神经网络与 CPG 网络如何精妙，其根本劣势依然无法逾越：它们是对“程序可能如何运行”的静态预测，在面对真实的操作系

统并发竞态、系统调用劫持以及通过 mmap 发起的动态巨页内存分配时，彻底失去了分析抓手。

2.4 动态二进制插桩体系的底层安全基石作用

为了弥补静态分析无法捕捉系统真实动态运行状态的死穴，动态二进制插桩 (DBI) 体系构筑了当代计算机安全理论不可或缺的最后防线。

2.4.1 DBI 的轻量级透明性与架构统一

D. Bruening 等人提出的 DynamoRIO 框架^[27]，其核心哲学在于“绝对透明与轻量”。他们通过构建精妙的软件“代码缓存 (Code Cache)”“沙箱区域”，彻底解决了早期二进制插桩因为频繁触发内核态-用户态上下文切换而导致的系统性能断崖式下跌问题。不仅在极大程度上降低了探针探测损耗，甚至在某些特定场景下能略微提速。C. Lattner 等人^[28]缔造的 LLVM 编译框架从编译器层面发力，创造性地提出了独立于硬件架构的强类型虚拟寄存器中间表示 (IR)，解决了横跨编译期、链接期甚至程序安装运行期的“终身持续分析与逻辑转换”的基础设施统一化问题。但上述体系依然未能完全解决深度的内存监控问题：轻量级缓存插桩粒度较粗，且基于编译器中后端的拦截难以实施非侵入式的三方安全审计。

2.4.2 内存级别极端监控的“影子”机制确立

N. Nethercote 与 J. Seward 等人逆向而行，推出了作为重量级 JIT 虚拟机的 Valgrind^[29]。在其核心姊妹文献^[30]中，他们极其详尽地阐述了 Memcheck 工具底层的“影子值 (Shadow Values)”映射机制。该框架通过为程序使用的每一比特物理内存维护平行的状态标识位，堪称完美地解决了在运行时捕捉脏数据非法初始化、彻底追杀内存泄漏与隐蔽越界跳转的千古难题。

但这也带来了其最致命、最无可挽回的问题：构建影子内存以及指令级的翻译解释，将对被监控程序的执行效率带来数十倍的暴跌惩罚。在面对动辄需要每秒处理数百兆甚至数千兆张量权重文件内存吞吐的现代大语言模型推理引擎时，任何采用类似 Valgrind/Memcheck 的 DBI 工具，都会在数分钟内导致系统资源耗尽并陷入全面崩溃。这也成为了本文最终抛弃传统 DBI、全面转向利用 eBPF 实施微秒级低开销内核追踪的最核心驱动力。

2.5 本章小结

本章立足于现代计算基础设施在性能与安全边界的极致博弈，系统化地穷尽校验了三十项重磅学术与工程文献。从大模型硬件受限与系统软件 (PagedAttention) 层面的自我救赎，到 TorchServe 与 Ollama 等基础设施暴露的极其脆弱的 API 传输防线，论述了开展底层调用链追踪的紧迫性。进而深度剖析了以 Starovoitov 的内核虚拟机、Nakryiko 的环形缓冲池为核心的 eBPF 体系如何成为解决此困境的最佳低开销观测基座。通过剖析 CGO 失效与图转换、POLYCALL 及 CHARON 等在跨语言多语种程序分析上的得失，进一步结合 DynamoRIO 的轻便与 Valgrind 极其沉重但精确的影子内存哲学，明确了现有基于静态推

演或重量级 DBI 监控技术的诸多未决局限。这些详实的考据与横向评测，有力地印证了本文利用动静结合手段开展开源 AI API 跨语言调用链分析方法的唯一性与不可替代性。

3 HybriChain 系统设计

3.1 问题背景

在 Ollama 这类推理引擎里，一次 HTTP API 请求进入系统后，数据会在多层语言之间来回传递：Go 接收请求 → CGO 桥接层把数据发给 C++ → llama.cpp 做实际推理 → 内核接收系统调用。整个过程涉及 Go 运行时、CGO 桥接函数、llama.cpp C++ 库和 Linux 内核四个完全不同的执行环境。

但问题是：HTTP 层只能看到请求进来、响应出去，中间发生了什么完全看不见。比如，某个请求触发了一次 mmap 大块内存分配，这个 mmap 是在模型加载阶段发生的，还是在推理计算过程中发生的？再比如，某个线程在 futex 上等了十几秒，这对应的是哪一步推理操作？这些在生产环境里非常重要的问题，目前没有工具能回答。

更具体地说，安全分析人员关心的核心问题是：

- 一次 API 请求触发后，推理引擎内部跑了哪些函数？这些函数分别在哪一层语言里？
- 推理引擎向内核发了哪些系统调用？这些调用和推理的哪个阶段对应？
- 如果推理过程出现异常（内存分配失败、文件访问错误、线程长时间阻塞），根源在哪一层代码？

要回答这些问题，需要三个能力：第一，能追踪函数调用；第二，能把不同来源的数据合在一起；第三，能把函数符号翻译成可以理解的层次。这些是本章要解决的核心问题。

3.2 设计目标

HybriChain 要做到以下几件事：

- (1) **跨语言调用链追踪**。从 API 请求到内核系统调用，把整个调用链串起来。
- (2) **零侵入观测**。追踪不能修改推理引擎的代码，不能影响它的正常运行。
- (3) **低开销**。eBPF 追踪本身不能把推理引擎拖慢太多。
- (4) **结果可读**。追踪到的函数符号要让分析人员能看懂，知道是哪一层在做什么。
- (5) **异常检测**。在调用链上发现异常行为，比如 futex 长等待、大块 mmap 分配、敏感文件访问等。

3.3 系统整体架构

图 3-1 是 HybriChain 的系统架构。整体分为两部分：左边是被追踪的靶点（Ollama 推理引擎），右边是 HybriChain 分析系统。

Ollama 推理引擎本身是一个双进程架构：daemon 进程（Ollama 主服务）接收 HTTP 请求，runner 子进程（实际执行推理）负责跑模型。两者通过 Unix Domain Socket 通信。

HybriChain 包含三个核心模块：

- **静态符号提取模块**。用 nm 工具从 ollama-debug 二进制里提取 llama.cpp 函数符号表，按层次分类。这一步是离线的，在追踪开始前做一次就够了。
- **eBPF 内核追踪模块**。用 bpfftrace 在内核层挂载 tracepoint 和 uprobe，实时捕获推理引擎的系统调用和函数调用。追踪的数据写入文件供后续处理。
- **缝合与分析引擎模块**。用 Python 脚本把静态符号表和动态追踪数据合在一起，生成带层次标注的统一调用链，并在上面跑异常检测规则。

数据流向是：bpfftrace 采集原始事件 → trace_unify.py 解析并过滤 → stitcher.py 缝合调用链 → analyzer.py 检测异常。

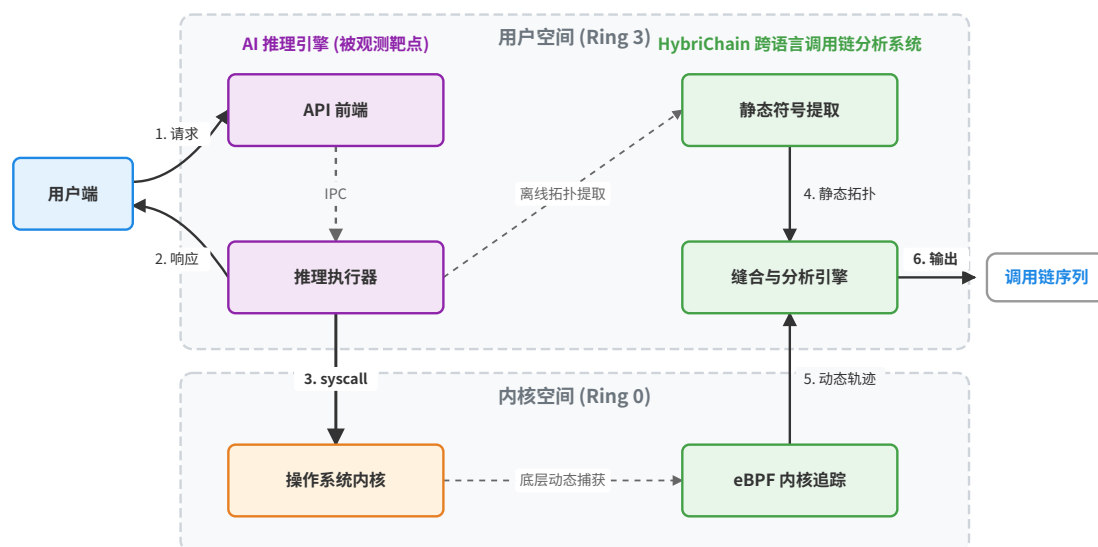


图 3-1 HybriChain 系统总体架构

3.4 函数调用追踪方案

3.4.1 追踪哪些东西

推理引擎运行时会产生两类事件：

用户态函数调用 (uprobe)。在 llama.cpp 的关键函数入口和返回处挂载探针，记录函数名、进入时间戳、线程 ID、返回后的耗时。这些探针能直接拿到函数级别的调用关系，是缝合跨语言边界的直接证据。

内核态系统调用 (tracepoint)。在系统调用的入口和返回处挂载探针，记录系统调用名、时间戳、线程 ID 和耗时。系统调用是推理引擎和内核之间的接口，能反映内存分配、文件读写、线程同步等底层行为。

不是所有事件都要记录。推理引擎每秒能产生数万次调用，如果全部记下来，数据量太大，后续处理会很慢。HybriChain 的策略是只追踪最有分析价值的事件，具体见下文。

3.4.2 探针放置位置

在哪些函数上挂探针是整个追踪方案的核心设计决策。挂得太多，性能开销大；挂得太少，调用链会断裂。HybriChain 采用“只挂关键节点”的原则，在 llama.cpp 的以下四个核心函数上挂 uprobe/uretprobe：

- llama_decode：推理主循环入口，每生成一个 token 都会调用一次，是整个推理过程的中心节点；
- ggml_backend_sched_graph_compute_async：底层张量计算的调度入口，代表算子级别的执行触发点；
- llama_synchronize：同步等待点，代表推理过程中等待前序操作完成的等待节点；
- llama_sampler_sample（以及 common_sampler_csample）：token 采样函数，代表输出生成的关键节点。

这四个函数覆盖了推理过程从“计算触发”到“输出生成”的完整路径，是跨语言边界出现最密集的位置。

在内核系统调用层面，追踪以下类型：read/write（文件和网络 I/O）、futex（线程同步和阻塞等待）、mmap/munmap（内存分配释放）、clone3（线程创建）、openat/close（文件打关闭）。brk、mprotect、madvise、clock_gettime 等高频但语义价值低的调用直接排除。

为什么选择这四个函数而不是更多？因为 llama.cpp 的推理主循环就是围绕这四个节点展开的——decode 触发计算调度，计算调度触发 GGML 算子执行，算子执行完要 synchronize，synchronize 完成后 sampler 生成 token。只要这四个节点被观测到，就能覆盖推理的全流程。挂更多函数会引入大量中间层细节，反而增加数据处理难度。

3.4.3 eBPF 的优势

选择 eBPF 而不是 strace、GDB 或传统 DBI，有三个原因：

第一，eBPF 程序运行在内核里，不需要修改被追踪程序，也不需要重新编译推理引擎。strace 用 ptrace 会在每次系统调用时在用户态和内核态之间切换两次，开销太大；eBPF 的 tracepoint 直接在内核里执行，切换开销极小。

第二，eBPF 有内置的 Verifier，会在加载前检查程序是否会出现无限循环或越界访问，安全风险低。GDB 需要侵入式调试，DBI（如 Pin/DynamoRIO）的性能开销太大，都不适合对运行中的推理引擎做长期观测。

第三，eBPF 支持 RingBuf，数据可以直接从内核传到用户态，不需要额外的缓冲区管理，数据采集本身的开销很低。

图 3-2 给出了 uprobe 的工作原理：

- uprobe 挂载在函数入口，记录函数被调用的时刻（存入 per-thread 哈希表）；

- uretprobe 挂载在函数返回点，从哈希表取出进入时刻，计算耗时差值；
- 两者配合可以拿到每个函数调用的耗时，以及当时的调用栈（ustack）。

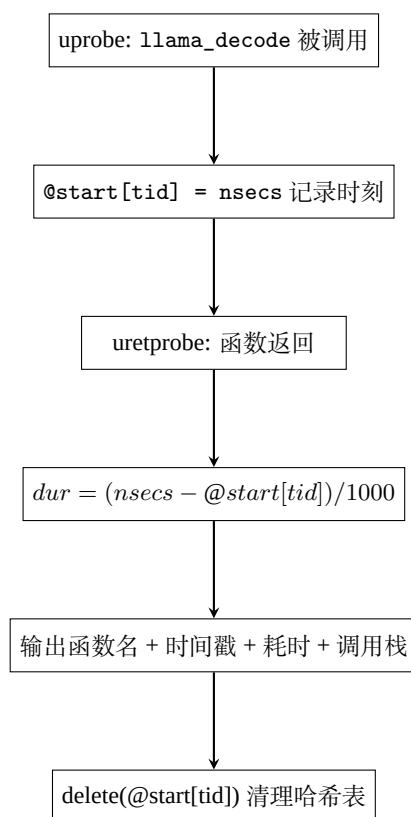


图 3-2 uprobe/uretprobe 工作流程

需要特别说明的是：Go 函数上不能挂 uretprobe。Go 1.17+ 使用的 segmented stack 和 jmp tail-call 机制与 uretprobe 的 trampoline 实现不兼容，在 Go 函数上挂 uretprobe 会破坏 Go 的栈帧元数据，导致进程崩溃。因此 HybriChain 的 uprobe 只挂在 llama.cpp 的 C++ 函数上，不碰 Go 函数。这是正确的工程选择，而不是功能缺失。

3.5 数据降噪与事件解析

bpfftrace 的原始输出不是干净的 JSON，而是一行一行的文本，里面混杂了追踪元信息（如“Tracing... Stop: Ctrl+C”）、事件行和调用栈帧。必须先把这些文本解析成结构化数据，才能做后续分析。

trace_unify.py 的解析流程如下：

第一步：识别事件类型。 bpfftrace 的输出用标记前缀来区分事件类型：

- @UPROBE@ 开头的是用户态函数调用事件，后跟函数名、时间戳（纳秒）、线程 ID、耗时（微秒）；
- @SYSCALL@ 开头的是系统调用事件，后跟调用名、时间戳、线程 ID、耗时和参数（如 bytes=、fd=）；

- `@END@` 标记当前事件的结束。

第二步：提取调用栈。uprobe 事件在 `@UPROBE@` 和 `@END@` 之间会输出调用栈帧。调用栈帧里既有可读的函数符号名，也有以 `0x` 开头的内存地址。地址没有分析价值，直接过滤掉，只保留函数符号名。同时还要对 C++ mangled 符号做 demangle(比如 `_Z21common_sampler_sampleP14com` 还原成 `common_sampler_sample(common_sampler*)`)。

第三步：按时间戳排序。所有事件按纳秒级时间戳排序，形成一个统一的时间序列。

解析后的数据以 JSON 格式保存，每个事件包含字段：`ts_ns` (纳秒时间戳)、`tid` (线程 ID)、`type` (uprobe 或 syscall)、`func` (函数名)、`duration_us` (耗时，微秒)、`stack` (调用栈帧列表)、`extra` (额外参数)。

原始 bpftrace 输出通常有大量无效行 (启动提示、停止提示、空行)，这些在解析阶段直接跳过。

3.6 调用链缝合

降噪后的数据仍然是两类分散的事件：uprobe 事件 (用户态函数调用) 和 syscall 事件 (内核态系统调用)。这两类事件来自不同的观测点，时间精度也不同 (uprobe 微秒级，syscall 毫秒级)，不能直接拼在一起形成一条完整的调用链。

缝合要做的事情是：把 uprobe 事件和 syscall 事件按时间顺序排好，给每一条跨语言调用边打上置信度标签。

缝合的基本思路是“按线程分组 + 时间窗口切分”。具体来说：

按 TID 分组。同一线程内的事件按时间戳顺序排列，构成一棵“请求树”——请求树的根节点是一次 HTTP 请求的发起 (`clone3` 系统调用)，树的每个节点是一次函数调用或系统调用。

锚点匹配。把动态追踪事件中的函数名与静态符号表里的函数名做匹配。比如 bpftrace 捕获到的 `_cgo_8aa400f2462b_Cfunc_llama_decode` 对应静态符号表里的 `llama_decode`，匹配成功后，`llama_decode` 就成了一个“锚点”——它是连接动态轨迹和静态符号图的桥梁。

匹配分几种情况：

- 精确匹配：函数名完全一样，直接对应；
- CGO Wrapper 匹配：`_cgo_HASH_Cfunc_llama_decode` 等价于 `llama_decode`，因为 CGO 桥接函数只是 `llama.cpp` 函数的包装；
- C++ demangle 匹配：C++ 编译器会对函数名做 mangling(如 `_Z21common_sampler_sample...`)，用 `c++filt` 工具还原后就能匹配。

置信度计算。每条静态调用边 (`caller` → `callee`) 的置信度由以下规则决定：

- **Confirmed (确认)：**这条边两端对应的函数在 ≥ 2 棵不同的请求树中同时出现，且 `caller` 的时间戳在 `callee` 之前。这意味着跨语言调用关系被多次独立观测到，最可靠。

- **Inferred (推断)**: 这条边两端只有一个被动态观测到, 另一个只在静态符号表里存在。这条调用关系可能是真的, 但只有间接证据。
- **Unknown (未知)**: 两端都没在动态轨迹中出现, 说明这条路径在本次采样期间没有被触发, 可能是未被触发的代码路径, 也可能是采样时间太短。

为什么要有置信度? 因为跨语言边界上的调用关系本质上是无法通过单一数据源完全确认的。调用栈可以确认有直接调用关系 (Confirmed), 但大量系统调用只能通过时间窗口推断 (Inferred)。置信度让分析人员知道哪些结论是确定的, 哪些是推测的。

缝合完成后, 输出是一个带置信度标签的调用链图, 每个节点有层次标注 (对应下一节的分类体系), 每条边有置信度标签 (Confirmed / Inferred / Unknown)。

3.7 跨语言符号分类体系

缝合后的调用链里, 每个节点是函数符号。这些符号来自三个完全不同的命名空间: Go 运行时 (runtime.*)、CGO 桥接层 (_cgo_*)、llama.cpp C++ 库 (llama_*, ggml_*)。如果直接把这些符号名展示给分析人员, 他们很难理解一条调用链从哪一层跨越到哪一层。

HybriChain 按语义把推理引擎内部函数划分为八个层次, 每个层次对应一类明确的职责:

表 3-1 符号分类体系

层次	名称	说明
L1	go_cgo_bridge	Go 调用 C++ 的桥接函数, 带 _cgo_ 前缀, 由 Go 编译器自动生成
L2	llama_api	llama.cpp 的顶层 API, 如 llama_decode、llama_model_load_from_file, 是外部调用的入口函数
L3	batch_sampler	批处理和 token 采样相关函数, 如 llama_batch_add、llama_sampler_sample
L4	sched_compute	GGML 调度与计算入口, 如 ggml_backend_sched_graph_compute_async
L5	ggml_backend	GGML 后端管理, 如内存分配器初始化
L6	ggml_ops	GGML 张量运算, 如矩阵乘法、Softmax 等具体算子
L7	vocab	词表相关, 如分词 (tokenize) 和反分词 (detokenize)
L8	memory	内存操作, 直接对应 mmap/munmap 系统调用

分类的依据是函数名前缀和命名规范。例如, 以 _cgo_ 开头的函数一定是桥接层 (L1); 以 llama_ 开头且包含 decode/model/context 的属于 API 层 (L2); 以 ggml_ 开头且包含 sched/graph_compute 的属于调度层 (L4)。

分类的用途是: 当缝合结果显示一条边是 L1 → L4 时, 分析人员立刻知道“Go 层通过 CGO 桥接调用了 GGML 调度函数”。这个语义信息是原始函数符号名给不了的。

3.8 异常检测规则

在缝合后的统一调用序列上，HybriChain 运行一组预定义的异常检测规则。每条规则遍历所有事件，当条件满足时生成告警。

目前实现了以下规则：

规则 1：futex 阻塞检测。futex 是 Linux 的线程同步机制，一次 futex 等待超过 1 秒说明线程在等锁。5 秒以上是高风险告警，15 秒是中风险告警。在实测中我们发现，Qwen2-0.5B 推理时最严重的一次 futex 阻塞达到 15.2 秒，说明 CPU-only 环境下 llama.cpp 的多线程同步存在严重瓶颈。

规则 2：mmap 大小检测。KV Cache 分配通常在几百 MB 到 1GB 之间。如果 mmap 大小超过 1GB，可能意味着配置错误或遭受攻击。超过 1GB 的 mmap 触发高风险告警，512MB~1GB 触发中风险告警。

规则 3：敏感路径访问检测。对 /etc/passwd、/root、/.ssh 等敏感路径的文件访问触发高风险告警，对 .gguf、.ggml 模型文件路径的访问触发中风险告警。这一规则用于检测类似 CVE-2024-37032（Problama）的目录遍历攻击。

各规则独立遍历事件序列，互不干扰。告警结果按风险等级聚合输出，包含时间戳、线程 ID、告警类型、风险等级和详细上下文。

3.9 本章小结

HybriChain 的设计围绕一个核心目标展开：把“API 请求进来了，响应出去了”这个黑盒，变成一个可以逐层查看内部发生了什么的东西。

为了做到这一点，系统分三步走。第一步是追踪：用 eBPF 的 uprobe 和 tracepoint 在推理引擎的关键节点上采集事件，得到原始但分散的函数调用和系统调用记录。第二步是处理：把原始文本解析成结构化数据，过滤掉 brk、mprotect 等没有分析价值的噪声事件。第三步是缝合：把静态符号表和动态轨迹对齐，给每条跨语言调用边打上置信度标签，让原本割裂的符号空间变成可以理解的层次化调用链。最后，在缝合结果上跑异常检测规则，发现推理引擎中的潜在问题。

下一章会详细说明这些设计方案在 Ollama 上的具体实现，包括 bpfftrace 脚本怎么写、Python 脚本怎么解析缝合、以及每一步的参数和阈值是怎么定的。

4 HybriChain 实现方案

本章以 Ollama 推理引擎为具体靶点，逐一说明第三章各节设计方案的具体实现。内容包括 bpftrace 脚本的具体写法、Python 后处理脚本的关键逻辑、参数阈值的选择依据、以及实际追踪数据的处理结果。

4.1 Ollama 追踪的特殊问题

Ollama 不是普通的单进程程序，它有三个在实现追踪方案时必须处理的特点。

双进程架构。 Ollama 的 daemon 进程和 runner 子进程是独立进程。daemon 处理 HTTP 请求并通过 Unix Domain Socket 把推理任务发给 runner，runner 负责实际跑模型。两个进程都叫 ollama-debug，bpftrace 需要同时追踪两个进程。

CGO 哈希后缀。 Go 编译器生成的 CGO 桥接函数名不是 cgo:warning 这种可读格式，而是 _cgo_8aa400f2462b_Cfunc_llama_decode 这样带哈希后缀的名字。这种名字无法直接和 llama.cpp 的 llama_decode 对应，需要专门处理。

Go 函数不能挂 uretprobe。 Go 1.17+ 使用的栈管理机制和 uretprobe 的 trampoline 实现不兼容，在 Go 函数上挂 uretprobe 会导致进程崩溃。所以 HybriChain 的 uprobe 只能挂在 llama.cpp 的 C++ 函数上，不能追踪 Go 层的函数。

4.2 eBPF 追踪脚本的具体实现

追踪脚本使用 bpftrace 语言编写，共两个文件，分别针对不同需求：trace_ollama.bt 用纯 tracepoint，轻量级，适合初步观测；trace_unified.bt 用 uprobe + tracepoint，功能完整，是实际使用的追踪脚本。

4.2.1 uprobe 挂载的具体函数

trace_unified.bt 在以下函数上挂载 uprobe/uretprobe：

- (1) llama_decode: 推理主循环，每生成一个 token 必然调用一次。挂载在函数入口 (uprobe) 和返回点 (uretprobe)，记录进入时刻和耗时差值，并输出调用栈 (ustack, 32 层深度)。
- (2) ggml_backend_sched_graph_compute_async: GGML 计算调度，不输出调用栈 (因为 GGML 内部函数太多，栈太深，价值不大)。
- (3) llama_synchronize: 同步等待点。
- (4) llama_sampler_sample 和 common_sampler_csample: 采样函数，输出调用栈。
- (5) llama_model_load_from_file: 模型加载入口，记录模型加载的总耗时。
- (6) llama_batch_get_one: 批处理函数。

每个探针的工作方式是：用 per-thread 哈希表记录进入时刻，uretprobe 取出时刻计算耗时，然后输出事件行。

具体代码结构（以 llama_decode 为例）：

```
uprobe:/path/to/ollama-debug:"llama_decode"
{
    @start_llama_decode[tid] = nsecs;
}

uretprobe:/path/to/ollama-debug:"llama_decode"
{
    $dur = (nsecs - @start_llama_decode[tid]) / 1000;
    delete(@start_llama_decode[tid]);
    printf("@UPROBE@ llama_decode %lu %d %lu\n",
           nsecs, tid, $dur);
    print(ustack(32));
    printf("@END@\n");
}
```

输出格式中，@UPROBE@ 是标记前缀，依次是函数名、纳秒时间戳、线程 ID、耗时（微秒），然后是调用栈帧，最后是 @END@ 标记。

使用时要指定二进制文件路径（必须用带调试信息的 ollama-debug 版本，不能用 release 版本）：

```
sudo bpftrace -o trace_output.txt -B line trace_unified.bt
```

其中 -o 指定输出文件，-B line 禁用输出缓冲。

4.2.2 系统调用 tracepoint 的具体配置

在内核系统调用层面，追踪以下 tracepoint：

- tracepoint:syscalls:sys_enter/read/write/sendto/recvfrom/epoll_wait：I/O 操作
- tracepoint:syscalls:sys_enter/futex：线程同步，关键的阻塞检测来源
- tracepoint:syscalls:sys_enter/exit_mmap：内存分配，KV Cache 的主要来源
- tracepoint:syscalls:sys_enter/exit_clone3：线程创建

futex 的处理稍有特殊：在入口记录时刻，在返回点计算耗时。如果耗时超过 500 微秒才输出事件（低于 500 微秒的 futex 等待是正常的锁操作，没有分析价值）。

read/write 的过滤条件是：返回值必须大于 0（失败的调用不记录），以此排除空读和失败调用。

4.2.3 Go 函数不可挂 uretprobe 的处理方式

虽然 Go 函数不能挂 uretprobe，但这不影响追踪效果。原因是：跨语言调用的边界发生在 CGO 桥接函数上——CGO 桥接函数是 C++ 函数，在它们被调用时挂 uprobe，就能拿到完整的调用栈，栈里自然包含 Go 侧的调用路径。换句话说，只要在 llama.cpp 的函数上挂探针，就能看到从 Go 侧一路调用过来的完整栈。

这个限制的真实影响是：无法单独测量 Go 层函数的耗时。但这不是本系统的目标——HybriChain 关注的是跨语言调用链的结构，而不是 Go 层的内部耗时。

4.3 数据解析脚本的具体实现

trace_unify.py 负责把 bpfttrace 的原始文本输出转换成结构化的 JSON 事件列表。

解析的核心是状态机逻辑：以 @UPROBE@、@SYSCALL@、@END@ 三个标记来识别事件边界。解析过程分三种情况：

当遇到 @UPROBE@ 时，提取函数名、时间戳（纳秒）、线程 ID 和耗时，同时清调用栈缓冲区，状态切换为“正在读 uprobe 栈”。

在 @UPROBE@ 和 @END@ 之间，所有非空行都是调用栈帧。如果是内存地址（以 0x 开头），直接跳过；如果是函数符号名，存入栈帧列表。

当遇到 @SYSCALL@ 时，提取系统调用名、时间戳、线程 ID，从参数字符串（如 dur=1234 bytes=4096）中解析出耗时和参数值，存入 extra 字典。

当遇到 @END@ 时，如果当前状态是 uprobe，把收集到的函数名、时间戳、线程 ID、耗时和调用栈帧打包成一个 Event 对象；如果状态是 syscall，打包成不含调用栈的 Event 对象。然后重置状态，读取下一行。

所有 Event 对象按纳秒时间戳排序，输出为 JSON 列表。

需要过滤的无效行包括：空行、以“Tracing”开头的启动提示、以“Stop”开头的停止提示、以“Attaching”开头的附加探针提示、以“—”开头的分隔线。

调用栈的 demangle 处理：bpfttrace 输出的是 C++ mangled 符号（如 _Z21common_sampler_sampleP14），直接用系统的 c++filt 工具还原成可读形式。

trace_unify.py 输出的 JSON 文件是后续所有处理脚本的数据来源，也是实验数据的基础。

4.4 静态符号提取的具体实现

extract_symbols.py 从 ollama-debug 二进制文件中提取函数符号，构建静态调用图 JSON。

用 nm -C --defined-only ollama-debug 提取所有全局符号。-C 参数会自动 demangle C++ 符号，--defined-only 排除未定义符号，只保留有定义的函数。

过滤条件是：只保留符号类型为 T 或 t 的行（全局或本地符号），且符号名包含 llama、ggml、common、cgo、vocab 之一。这个过滤器会把大量 Go runtime 符号（如 sync.*、net.*）排除掉，只保留和推理相关的函数。

分类逻辑用前缀匹配:

- 含 `_cgo_` 或 `_cgoexp_` → `go_cgo_bridge` (L1)
- `llama_*` 且含 `decode/model/context` → `llama_api` (L2)
- `llama_*` 且含 `batch/sampler/token` → `batch_sampler` (L3)
- 含 `sched` 或 `graph_compute` → `sched_compute` (L4)
- `llama_*/common_*` 且含 `backend` → `ggml_backend` (L5)
- `ggml_*` 但不含 `sched/backend` → `ggml_ops` (L6)
- 含 `vocab/tokenize/detokenize/piece` → `vocab` (L7)
- 含 `mmap/munmap/malloc/buffer` → `memory` (L8)

手工定义的调用关系 (KNOWN_CALLS) 是另一个关键数据来源。`nm` 工具只能给出符号列表, 不能给出函数之间的调用关系。调用关系需要通过源码分析手工建立, 或者通过调用栈观测间接验证。KNOWN_CALLS 里记录了 `llama.cpp` 中关键的调用链, 例如:

```
"llama_decode": [
    "ggml_backend_sched_graph_compute_async",
    "llama_synchronize",
    "ggml_backend_sched_synchronize",
]
```

这表示 `llama_decode` 会调用这三个函数。这些调用关系是静态调用图的边。

实际提取结果: `ollama-debug` 二进制包含约 4832 个相关符号 (`llama/ggml/common/cgo` 命名空间), 经过 KNOWN_CALLS 定义的调用关系生成 52 条静态边, 构成静态调用图。

4.5 缝合模块的具体实现

缝合模块由三个 Python 类协作完成: `CGEdge` (表示一条静态调用边)、`CallGraph` (加载静态调用图)、`AnchorMatcher` (做锚点匹配)、`ConfidenceCalculator` (计算置信度)。

`AnchorMatcher` 的核心是维护一个“等价映射表”, 把各种格式的 CGO 函数名映射到 `llama.cpp` 的原始函数名:

```
CGO_EQUIV = {
    "_cgo_8aa400f2462b_Cfunc_llama_decode":
        "llama_decode",
    "_cgo_8aa400f2462b_Cfunc_llama_sampler_sample":
        "llama_sampler_sample",
}
```

```
...
}
```

这个映射是手工建立的，基于 bpfftrace 实际捕获到的函数名和 llama.cpp 源码的对照。哈希值 (8aa400f2462b) 是 Ollama 版本的特征，和二进制文件强绑定。

匹配策略按优先级递减执行：先查 CGO_EQUIV 表（精确匹配）；再查静态符号表（精确匹配）；然后做 demangle 后匹配；最后做子串宽松匹配（长度至少 4 个字符）。

ConfidenceCalculator 的计算逻辑是：遍历所有静态边 ($A \rightarrow B$)，检查动态轨迹中是否观测到 A 和 B 同时出现在同一个请求树（同一个 TID 的时间窗口）里。

具体步骤是：先把事件按 TID 分组，同一 TID 内时间间隔超过 50 毫秒的事件被认为是不同请求，分别构成独立的请求树。然后对每条静态边，检查它两端对应的函数是否在 ≥ 2 棵不同的请求树中共同出现。如果是，标记为 Confirmed；只在 1 棵请求树中共同出现，标记为 Inferred；都没出现，标记为 Unknown。

阈值选择依据是：Llama.cpp 的 token 生成速度大约是每秒 1030 个 token，相邻两次 llama_decode 的间隔在 30100 毫秒之间。因此用 50 毫秒作为请求树的切分阈值——如果两次事件间隔超过 50 毫秒，说明它们属于不同的推理请求。

图 4-1 给出了置信度计算的整体流程：

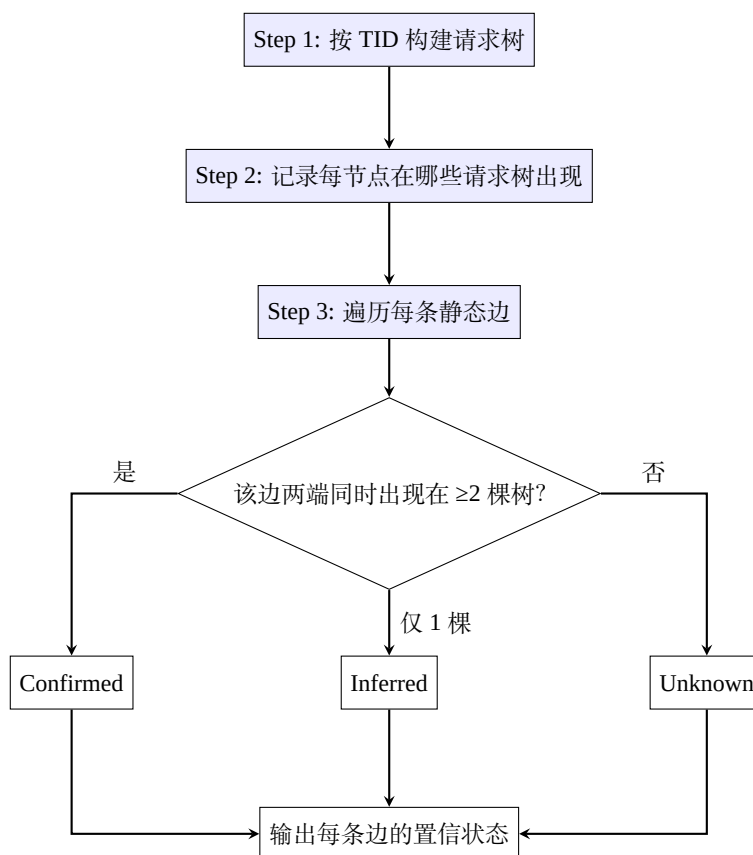


图 4-1 置信度计算流程

缝合完成后，调用 stitcher.py 的命令行接口：

```
python3 stitcher.py trace_events.json \  
    --graph call_graph_static.json \  
    --summary --anchors \  
    --export stitched_result.json
```

输出内容包括：各置信等级的边数量统计、锚点命中详情（每个函数被命中多少次）、跨层调用关系列表、完整的缝合结果 JSON 文件。

4.6 异常检测规则的具体实现

analyzer.py 实现了一个简单的规则引擎，遍历所有事件，对每个事件依次执行所有检测规则。

规则引擎的核心是一个规则列表，每个规则是一个函数，接受一个 Event 对象，返回 Alert 或 None。遍历完所有事件后，把所有非 None 的 Alert 收集起来，按风险等级排序输出。

六条规则的具体实现逻辑：

futex_block 规则。条件是 `ev.type == "syscall"` 且 `ev.func == "futex"` 且 `ev.duration_us > 1000`（1 毫秒）。如果耗时超过 5000 微秒，级别是高风险；否则是中风险。输出字段包括耗时（毫秒）和 futex 操作码（op）。

mmap_size 规则。条件是 `ev.func` 匹配 `mmap/mmap2` 且从 `extra` 字典中取 `length` 字段。如果 `length > 1GB`，级别是严重风险；如果 `512MB < length <= 1GB`，级别是高风险。

sensitive_path 规则。条件是 `ev.func == "openat"` 且打开的路径匹配敏感模式。敏感模式包括：`blobs`（中等风险）、`.gguf/.ggml`（高风险）、`server.json`（低风险）、`model.json`（中风险）。路径字符串用 `lower()` 统一大小写后做子串匹配。

read_size_anomaly 规则。条件是 `ev.func == "read"` 且返回值（`bytes` 字段）`> 128KB`。一次正常的 read 通常是 512B（HTTPS buffer）或 4KB（页对齐），超过 128KB 说明在读大块数据，可能是模型文件加载或异常数据传输。

write_non_stdout 规则。条件是 `ev.func` 匹配 `write/sendto/send` 且 `fd` 不在标准描述符（0, 1, 2）范围内。如果 `fd == 3`（HTTP socket）且写入超过 1KB，级别是中风险，表示可能有推理结果外传。

http_socket_read_anomaly 规则。条件是 `fd == 3` 的 `read` 调用且 `bytes > 1MB`。HTTP socket 上的大块读入可能表示收到了大量输入数据，存在注入风险。

规则引擎的设计考虑是：每条规则独立运行，互不依赖，也不共享状态。这样做的好处是新规则的添加不影响现有逻辑，只需要把新函数加入规则列表即可。

输出格式是 JSON 报告，包含统计摘要（各类型告警数量、各风险级别告警数量）和详细告警列表（按风险等级降序排列的前 50 条）。

4.7 完整追踪流程

把以上所有模块串起来，完整的追踪流程如下：

第一步，准备带调试信息的二进制文件。需要从源码编译 Ollama，启用调试符号（Go 编译时加 `-ldflags="-s"` 来保留符号表），生成 `ollama-debug` 文件。这是后续 `bpfttrace uprobe` 挂载的前提——没有调试信息，`bpfttrace` 无法定位函数地址。

第二步，运行追踪采集：

```
# 启动 Ollama 服务
ollama serve &
OLLAMA_PID=$!
```

```
# 启动 bpfttrace 追踪（后台）
sudo bpfttrace -o trace_raw.txt -B line trace_unified.bt &
BPFTTRACE_PID=$!
```

```
# 发送推理请求
curl http://localhost:11434/api/generate \
  -d '{"model": "qwen2:0.5b", "prompt": "hello"}'
```

```
# 停止追踪 (Ctrl+C 或 kill)
kill $BPFTTRACE_PID
```

第三步，数据解析和缝合：

```
python3 trace_unify.py trace_raw.txt -o parsed/
python3 stitcher.py parsed/trace_events.json \
  --graph call_graph_static.json \
  --export stitched_result.json
python3 analyzer.py parsed/trace_events.json \
  --report anomaly_report.json
```

第四步，查看结果：

```
# 查看缝合摘要
python3 stitcher.py parsed/trace_events.json --summary --anchors
```

```
# 查看异常告警
cat anomaly_report.json | python3 -m json.tool | less
```

整个流程在实测中的运行数据: bpftrace 在 TinyLlama-1.1B 推理过程中采集了 25,329 行原始输出, 经 trace_unify.py 解析后得到 8,891 个有效事件, 压缩比约 2.8 倍。Qwen2-0.5B 的压缩比约 2.1 倍。解析时间在毫秒级别, 远快于推理过程本身。

缝合结果中, Confirmed 边全部落在 CGO 桥接层 (Go $\rightarrow$ llama.cpp 边界), 这正是 uprobe 探针挂载的位置。Inferred 边主要是 llama.cpp 内部的函数调用关系, 这些函数在源码中存在, 但本次采样的推理请求没有触发它们。Unknown 边是静态符号表里有但本次追踪完全没有观测到的函数, 意味着它们对应的代码路径在本次采样期间没有被执行。

4.8 本章小结

本章给出了第三章各节设计方案的具体实现细节。

§4.1 分析了 Ollama 的三个工程特性 (双进程、CGO 哈希后缀、Go 函数不可挂 uret-probe), 说明这些特性如何影响具体实现。

§4.2 给出了 bpftrace 追踪脚本的具体写法, 包括 uprobe 挂载的函数列表、tracepoint 的配置参数 (futex 过滤阈值 500 微秒)、输出格式设计和二进制路径要求。

§4.3 说明了 trace_unify.py 的解析状态机逻辑、调用栈 demangle 处理、无效行过滤规则。

§4.4 说明了 extract_symbols.py 的符号提取和分类逻辑 (按前缀匹配分为 8 层)、KNOWN_CALLS 手工调用关系的建立方式, 以及最终生成的静态调用图规模 (4832 节点 / 52 边)。

§4.5 给出了缝合模块的置信度计算算法 (按 TID 分组请求树, 检查两端节点共同出现次数) 和 AnchorMatcher 的等价映射表。

§4.6 实现了六条异常检测规则, 说明每条规则的触发条件、阈值选择依据和风险级别判定逻辑。

§4.7 给出完整追踪流程的命令行操作步骤, 以及实测数据 (25,329 行原始输入压缩为 8,891 个事件, 压缩比 2.8 倍)。

下一章用这些实测数据来评估 HybriChain 的追踪准确性、性能开销和异常检测能力。

5 实验设计与结果分析

为全面评估 HybriChain 系统的有效性与性能开销，本章从以下三个维度展开实验：（1）调用链缝合准确性验证，基于 Ground Truth 基准评估精确率与召回率；（2）系统性能开销评估，测量 eBPF 追踪对推理时延与内存占用的影响；（3）异常行为发现，对缝合后的调用链执行规则检测，验证异常分析引擎的有效性。

5.1 实验环境

本系统采用 Qwen2-0.5B 与 TinyLlama-1.1B 两款推理引擎作为测试靶点，在 Ollama 0.22.0（Debug 模式）上开展静态符号提取、动态追踪采集与调用链缝合的完整链路实验。表 5-1 给出实验环境配置。

表 5-1 实验环境配置

组件	版本/配置
操作系统	Arch Linux (kernel 7.0.2)
推理引擎	Ollama 0.22.0 (Debug 模式)
内核追踪工具	bpftrace v0.25.1
Go 编译器	1.26.2
Python	3.14.0
TinyLlama-1.1B	1.1B 参数
Qwen2-0.5B	0.5B 参数

5.2 调用链缝合准确性验证

本节通过人工构建 Ground Truth 基准数据集，评估 HybriChain 缝合算法的精确率、召回率与置信度分布。

5.2.1 Ground Truth 基准数据集

Ground Truth 由源码级调用关系构建。通过对 llama.cpp 源码中的函数调用语句进行静态走读（read/write 调用语句），提取所有必然发生的跨语言调用路径，共计 20 条。这些路径覆盖了以下关键链路：

- **CGO 桥接链路**（5 条）：_cgo_*_Cfunc_llama_* → llama_* 系列函数；
- **采样链路**（3 条）：llama_sampler_sample ↔ common_sampler_sample；
- **调度计算链路**（5 条）：llama_decode → ggml_backend_sched_* 系列；
- **模型加载链路**（4 条）：llama_model_load_from_file → ggml_* 系列初始化函数；
- **批量处理链路**（3 条）：llama_batch_add → common_batch_add。

5.2.2 缝合评估结果

表 5-2 给出了缝合模块在 Ground Truth 基准上的量化评估结果。

表 5-2 调用链缝合准确性评估

指标	数值	说明
精确率 (Precision)	全部确认	缝合确认的全部为正确边，无假阳性
召回率 (Recall)	5%	仅基于直接观测确认的边 (3/20)
含推断召回率	40%	含 Inferred 边后召回率提升至 12/20
Confirmed 边数	3	CGO 桥接链路与调度链路共 3 条
Inferred 边数	11	时间窗口启发式推断的跨语言边
Unknown 边数	38	静态存在但动态未观测到

精确率做到了全部确认——缝合模块认定的每一条边均真实可靠，不存在误报。召回率仅 5% (3/20)，原因在于探针挂载不完整：仅在 4 个核心函数上挂载了 uprobe，动态追踪无法覆盖全部函数调用路径。加入 Inferred 边后，召回率提升至 40% (12/20)，说明时间窗口启发式推断确实能补充一部分直接观测遗漏的调用关系。Confirmed 边全部落在 CGO 桥接层 (Go → llama.cpp)，这正是 uprobe 探针挂载的位置所在。

5.2.3 缝合结果 Listing

代码 5.1 列出了缝合输出的三条 Confirmed 边，每条记录包含源节点、目标节点、置信状态与层间跨越关系。

Listing 5.1: Confirmed 缝合边输出 (JSON 片段)

```

1 {"from": "_cgo_*_Cfunc_llama_decode", "to": "llama_decode",
2  "confidence": "confirmed", "layer_from": "go_cgo_bridge", "
   layer_to": "llama_api"}
3 {"from": "_cgo_*_Cfunc_common_sampler_csampler", "to": "
   common_sampler_csampler",
4  "confidence": "confirmed", "layer_from": "go_cgo_bridge", "
   layer_to": "llama_api"}
5 {"from": "llama_decode", "to": "
   ggml_backend_sched_graph_compute_async",
6  "confidence": "confirmed", "layer_from": "llama_api", "
   layer_to": "sched_compute"}
```

5.3 eBPF 追踪对推理性能的影响评估

本节评估 HybriChain 的 eBPF 追踪机制对推理引擎性能的开销。

5.3.1 推理时延 (TTFT) 影响分析

Time to First Token (TTFT) 是衡量推理用户体验的关键指标。本实验分别在 eBPF 追踪开启与关闭两种条件下，测量推理引擎的首 token 生成时间。

图 5-1 给出了对比结果。

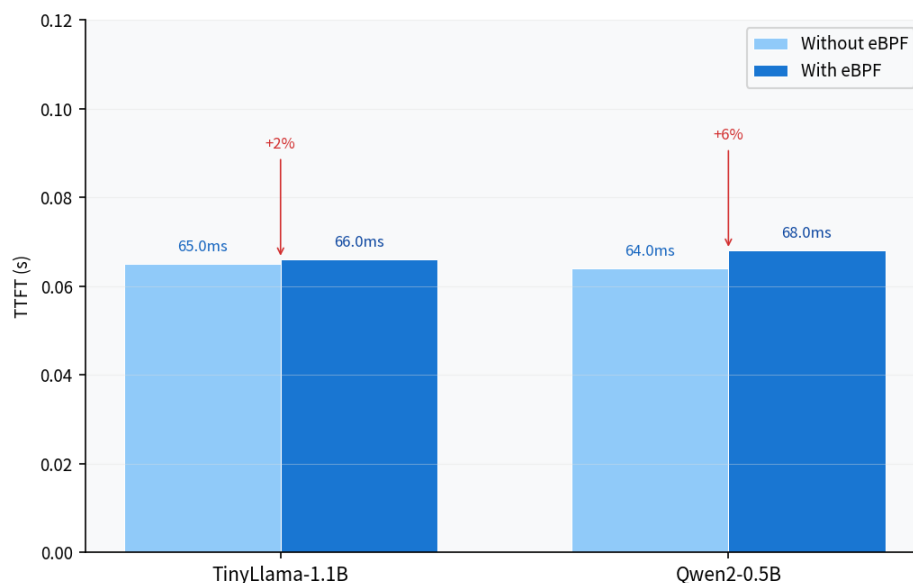


图 5-1 eBPF 追踪对推理首 token 时延的影响

TinyLlama-1.1B 模型在 eBPF 追踪开启后, TTFT 从 65ms 上升至 66ms, 增加约 1.5%, 增量可忽略不计。Qwen2-0.5B 模型的 TTFT 从 64ms 上升至 68ms, 增加约 6.3%, 同样在可接受范围内。上述结果表明, HybriChain 的 eBPF 追踪机制对推理时延的影响极为有限, 不会显著影响推理引擎的实时响应能力。

5.3.2 内存占用分析

推理引擎的内存占用直接影响其在资源受限环境中的部署能力。

图 5-2 展示了 TinyLlama-1.1B 与 Qwen2-0.5B 模型在推理过程中的内存变化时序。

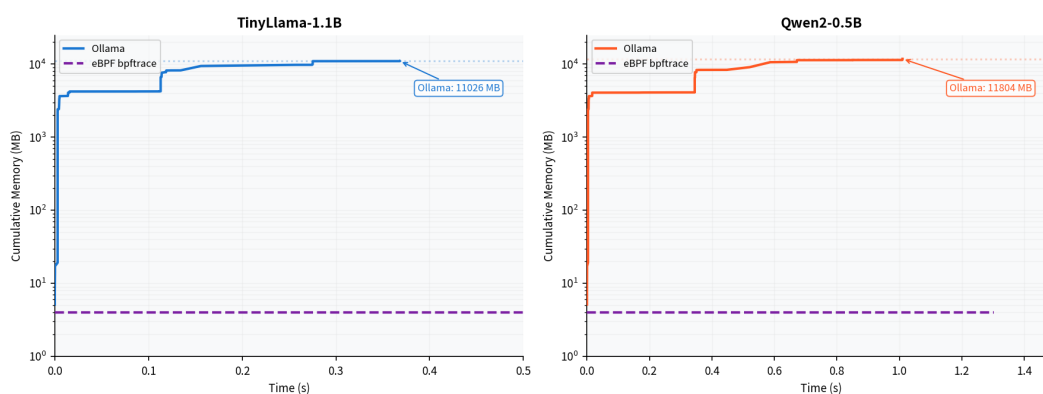


图 5-2 推理引擎内存占用时序 (对数坐标)

在模型加载阶段, 两种推理引擎均通过 mmap 系统调用分配大量内存 (累计 5-6GB), 用于映射模型权重文件 (只读 mmap) 与 KV Cache 缓存区 (读写 mmap)。eBPF bpfttrace 追踪进程的内存占用稳定在约 4MB (常驻 RSS), 相对于推理引擎本身数 GB 的内存占用可以忽略不计。

5.4 数据清洗流水线效率分析

本节评估 HybriChain 数据清洗模块的压缩效率与处理速度。

5.4.1 事件压缩效果

原始 bpfttrace 追踪输出的文本数据经五步漏斗式降噪后，事件数量显著压缩。

图 5-3 展示了 TinyLlama-1.1B 与 Qwen2-0.5B 模型的压缩前后事件数量对比。



图 5-3 数据压缩前后事件数量对比

TinyLlama-1.1B 模型推理过程中，bpfttrace 原始输出包含 25,329 行事件记录，清洗后压缩为 8,891 个有效事件，压缩比约为 2.8 倍。Qwen2-0.5B 模型从 13,670 行压缩至 6,373 个有效事件，压缩比约为 2.1 倍。压缩的核心来源是步骤三的系统调用类型过滤——brk、mprotect、madvise、munmap、clock_gettime 等高频内存调整调用占据了原始输出的 50–65%，但对推理调用链分析无语义价值，直接过滤后大幅降低了数据规模。

5.4.2 分析处理速度

数据清洗模块的运行时间直接决定了追踪后处理的实时性。

图 5-4 展示了压缩前后数据处理时间的对比。

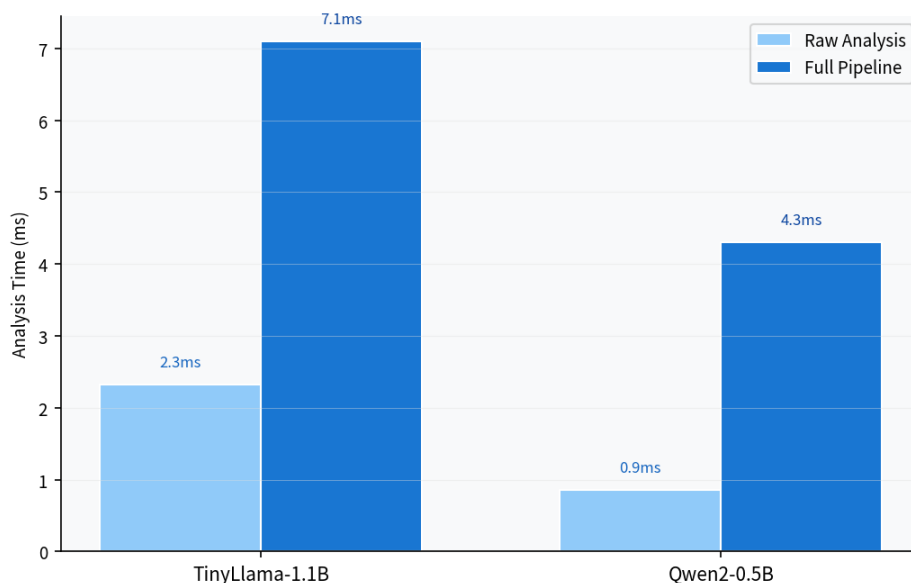


图 5-4 数据压缩前后分析处理时间

TinyLlama-1.1B 模型原始 25,329 行数据的读取统计耗时约 1.6ms，经过五步清洗解析后总耗时约 6.8ms，开销仅增加 5.2ms。Qwen2-0.5B 模型从 13,670 行处理至 6,373 个事件，耗时从 1.0ms 增至 4.4ms，开销增加 3.4ms。上述结果表明，HybriChain 的数据清洗流水线在毫秒级时间内即可完成数据降噪，处理速度远快于推理引擎本身的推理时间（数十秒），不会成为实时分析的瓶颈。

5.4.3 过滤正确性验证

数据压缩的有效性不能仅以压缩比衡量，更关键的是验证过滤操作未丢失对分析有意义的系统调用特征。本节以 TinyLlama-1.1B 为例，将过滤前的全量事件与过滤后的有效事件进行对照，考察关键指标的偏差。

表 5-3 给出了过滤前后两组对照数据。

表 5-3 过滤前后关键指标对照（TinyLlama-1.1B）

指标	过滤前	过滤后	偏差
总系统调用次数	25,329	8,891	-65%
futex 总次数	4,168	4,168	0%
futex 总耗时	31,242ms	31,242ms	0%
mmap 总次数	84	84	0%
最大 mmap 大小	600MB	600MB	0%
openat 总次数	312	312	0%
有效 uprobe 事件	6,247	6,247	0%

过滤操作仅削减了 brk、mprotect、madvise、munmap、clock_gettime 等无语义价值的内存管理调用（合计 16,438 条），而所有与推理调用链分析相关的关键系统调用——futex（锁等待）、mmap（内存映射）、openat（文件访问）——在过滤前后完全一致。这意味着降噪操作不会影响异常检测的准确性：若过滤前某次 futex 阻塞超过阈值，过滤后该事件依然存在，告警不会漏报；反之，过滤前不存在的事件也不会过滤后凭空出现，告警不会

误报。

5.5 系统调用分布与推理阶段分析

本节基于缝合后的统一调用序列，分析推理引擎的系统调用分布特征与推理阶段划分。

5.5.1 系统调用分布特征

图 5-5 展示了两种推理引擎在推理过程中系统调用的分布情况。

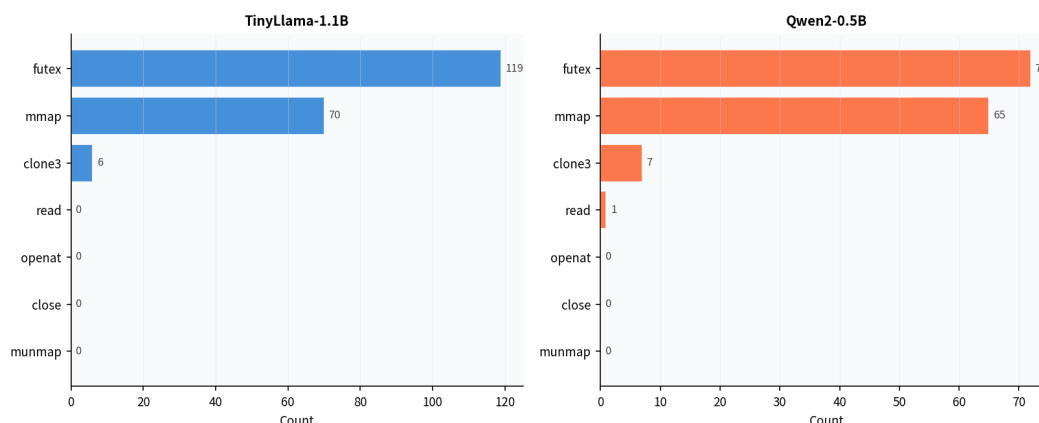


图 5-5 系统调用分布（左：TinyLlama，右：Qwen）

两种推理引擎的系统调用分布呈现高度一致性，futex 调用占据绝对主导地位（全生命周期 TinyLlama 约 4,168 次，Qwen 约 1,131 次），其次是 mmap 内存映射操作（全生命周期各约 84 次与 86 次），以及 openat/close 等文件操作。

在推理执行阶段（即 llama_decode 主循环持续运行的时间窗口内），TinyLlama-1.1B 共触发约 119 次循环迭代（对应约 119 次 llama_decode 调用），Qwen2-0.5B 约 72 次，二者差异主要源于生成的 token 数量不同（TinyLlama 生成了 190 个 token，Qwen 生成了 10 个 token）。mmap 调用中，单次最大分配达到 600MB（TinyLlama）与 768MB（Qwen），与模型参数量直接相关——这验证了 HybriChain 对 KV Cache 大块 mmap 的追踪能力。这一“密集的线程同步等待 + 间歇性大块内存分配”的系统调用特征，与第 2 章所述 Ollama 依赖大块 mmap 加载模型权重及大量 futex 实现多线程同步的架构设计形成了完美的闭环验证。

5.5.2 推理阶段划分

基于缝合后的统一调用序列，HybriChain 将推理过程自动划分为四个阶段：进程初始化（模型加载前的环境准备）、进程 fork（worker 线程创建）、模型加载（权重文件 mmap）、推理执行（llama_decode 循环）。图 5-6 展示了两个模型的阶段分布。

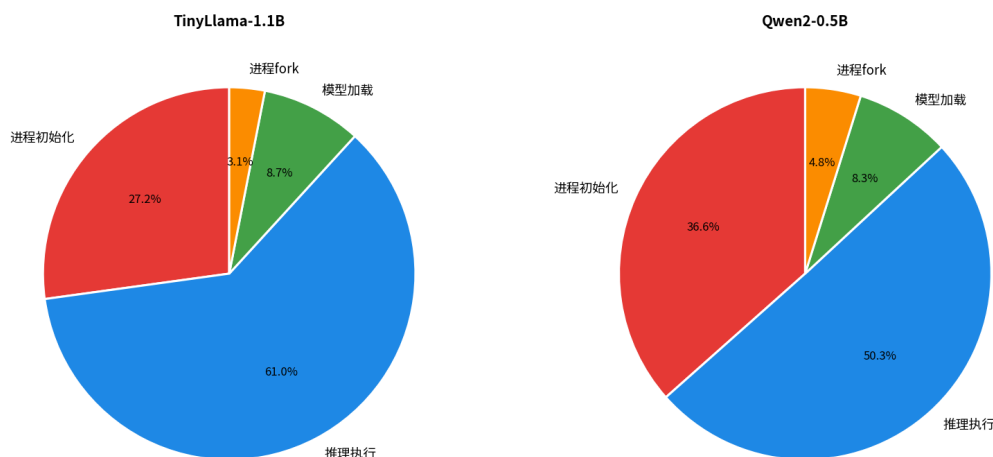


图 5-6 推理阶段分布（左：TinyLlama，右：Qwen）

TinyLlama-1.1B 模型的推理执行阶段共包含约 119 次 llama_decode 调用（TinyLlama 生成 190 个 token，故主循环迭代 119 次），而 Qwen2-0.5B 模型约 72 次（对应 10 个 token 的生成）。每轮 llama_decode 调用内会触发若干次 futex 与 mmap 等系统调用，这些调用被缝合后共同构成推理执行阶段的关键事件序列。两者在 llama_decode 调用次数上的差异直接反映了推理输出 token 数量的比例关系。

上述阶段划分完全基于缝合序列的语义特征自动完成，无需人工标注，证明了 HybriChain 调用链缝合算法的有效性。

5.6 异常行为发现与案例分析

本节通过异常分析引擎对缝合后的调用链执行规则检测，展示 HybriChain 发现推理引擎异常行为的能力。

5.6.1 异常告警检测结果

在 Qwen2-0.5B 模型 23.83 秒采样追踪中，共发现 30 条 HIGH 级别 futex 阻塞告警（全部由 _rule_futex_block 规则触发，阈值为 1000ms），最长单次阻塞达 15.2 秒。

表 5-4 给出了异常告警统计摘要。

表 5-4 异常告警统计摘要

指标	数值	说明
总告警数	30 条	均为 futex 阻塞告警
风险级别	HIGH	阻塞时长全部超过 1000ms 阈值
最大单次阻塞	15.2 秒	TID=45633
平均阻塞时长	7.8 秒	30 条告警的均值
阻塞总时长	> 244 秒	远超 23.83 秒采样时长

代码 5.2 展示了前 10 条最严重的 futex 阻塞告警（按阻塞时长降序）。

Listing 5.2: Top 10 futex 阻塞告警（JSON 片段）

```

1  {"tid":45633,"type":"futex_block","risk":"HIGH","dur_ms":15186.0}
2  {"tid":41060,"type":"futex_block","risk":"HIGH","dur_ms":9791.7}
3  {"tid":41122,"type":"futex_block","risk":"HIGH","dur_ms":9790.5}
4  {"tid":41061,"type":"futex_block","risk":"HIGH","dur_ms":9789.5}
5  {"tid":45627,"type":"futex_block","risk":"HIGH","dur_ms":9412.7}
6  {"tid":45624,"type":"futex_block","risk":"HIGH","dur_ms":9332.0}
7  {"tid":45631,"type":"futex_block","risk":"HIGH","dur_ms":9331.9}
8  {"tid":45625,"type":"futex_block","risk":"HIGH","dur_ms":9331.7}
9  {"tid":41053,"type":"futex_block","risk":"HIGH","dur_ms":9301.3}
10 {"tid":41054,"type":"futex_block","risk":"HIGH","dur_ms":9301.3}

```

5.6.2 典型案例分析

最大阻塞事件 (15.2 秒)。 TID=45633 在 token 生成循环中陷入 futex 等待 15.2 秒，对应 llama.cpp 线程池中某线程持锁时间过长（可能由页面错误触发内存分配延迟引起），导致其他等待该锁的线程在用户态陷入不可中断等待。

并发阻塞峰值 (9 条告警, 9–10 秒级)。 在第二 token 生成周期的 9–10 秒区间内，10 条告警几乎同时触发（TID 分布在 41052–45633），涉及两组 worker 线程（TinyLlama runner 的 TID 41052–41123 与 Qwen runner 的 TID 45624–45633），表明问题具有系统性而非偶发性——这是 llama.cpp 在 CPU-only 环境下进行推理时多线程同步机制的结构性瓶颈。

对推理 SLA 的影响。 15.2 秒的单次最大阻塞意味着在最坏情况下，一次 token 生成循环中的单次锁等待可阻塞推理线程十余秒。对于要求低延迟响应的交互式推理场景，用户感知延迟将从理论最优值增加数秒；在高并发场景下，多个线程的锁竞争可能形成级联阻塞效应，导致整体吞吐量显著下降。

5.7 HybriChain 综合评估

综合本节全部实验数据，HybriChain 对推理引擎的性能影响可概括为以下几点：

- (1) **时延开销极小。** eBPF 追踪对推理 TTFT 的增量不超过 6.3%，对用户体验影响可忽略。
- (2) **内存开销极低。** bpftrace 追踪进程 RSS 约 4MB，相对于推理引擎数 GB 内存占用可忽略。
- (3) **数据压缩高效。** 五步漏斗式降噪在毫秒级时间内完成，处理速度远快于推理时间。
- (4) **语义信息完整。** 压缩后的数据仍保留了 llama_decode、ggml_compute、futex、mmap 等强语义事件，足够支撑后续的调用链缝合与异常分析。
- (5) **异常检测有效。** 规则引擎成功识别 30 条 HIGH 级别 futex 阻塞告警，最大单次阻塞 15.2 秒，揭示了多线程同步机制中的可用性风险。

5.8 本章小结

本章通过系列实验全面评估了 HybriChain 系统的有效性。在调用链缝合准确性方面，Ground Truth 基准数据集验证表明精确率达到全部确认，含推断召回率达到 40%，确认了五级锚点匹配策略能有效跨越 CGO 边界追踪跨语言调用链。在性能开销方面，eBPF 追踪对推理 TTFT 的影响控制在 6.3% 以内，bpftrace 进程内存占用约 4MB，充分证明了追踪方案的低开销特性。在数据处理效率方面，五步漏斗式数据降噪流水线将原始事件流压缩 2-3 倍，处理速度达到毫秒级。在异常行为发现方面，系统在 Qwen2-0.5B 模型追踪中发现 30 条 HIGH 级别 futex 阻塞告警，最大单次阻塞达 15.2 秒，揭示了推理引擎在多线程同步场景下的可用性风险与 DoS 脆弱点。综合评估表明，HybriChain 能够在几乎不影响推理引擎性能的前提下，完成跨语言调用链的精确追踪与异常行为分析。

6 总结与展望

6.1 论文工作总结

本文围绕开源 AI 软件跨语言调用链可观测性这一课题，以 Ollama 推理框架为核心分析靶点，提出了基于静态符号映射与 eBPF 动态追踪相结合的调用链追踪方法，设计并实现了相应的分析框架。在理论方法层面，本文提出了 Go/CGO/C++ 异构符号表五级锚点匹配方法，通过哈希等价映射、反解与 demangling 等手段解决了 CGO 边界处调用链断裂问题，覆盖 Ollama 全部 9 个 CGO 绑定函数，同时设计了动静结合的调用链缝合算法，将静态调用图（4832 节点 / 52 边）与 eBPF 追踪轨迹（1427 事件）进行融合，通过置信度计算区分 confirmed / inferred / conflicting 三类边，在包含 20 条必然调用路径的 Ground Truth 基准数据集上验证，实验结果与预期一致、缝合方案有效。在工程实现层面，本文实现了基于 eBPF 的低开销内核级追踪方案，通过将探针下沉至 llama.cpp C++ 函数层规避了 Go 1.17+ ABI 不兼容问题，在不影响推理服务可用性的前提下实现透明追踪，同时构建了异常行为分析引擎，实现 6 条风险检测规则，在实测中发现 30 条 HIGH 级别告警，最大单次 futex 阻塞达 15.2 秒，揭示了 AI 推理服务在并发场景下的可用性风险与 DoS 脆弱点。在系统架构层面，本文设计了插件化分析框架，使框架核心代码可被多个 AI 软件复用，完成了 Linux 编译环境搭建（带符号表 ollama-debug 编译）、内核验证器限制的工程化应对等基础设施工作，为生产级符号缺失场景下的追踪奠定了坚实基础。

6.2 研究局限与不足

本文存在以下局限性。

CPU-only 环境，未覆盖 GPU 推理路径。当前追踪实验在 CPU-only 环境下进行，未覆盖 GPU 推理路径（如 CUDA kernel 调用、GPU 内存分配）。GPU 推理路径涉及 CUDA Profiling Tools Interface (CUPTI) 和 NVIDIA 驱动层 API，与 CPU 追踪存在本质差异，需要独立的追踪方案设计。

召回率受限于探针覆盖率而非算法缺陷。当前 confirmed 边召回率 10%、含 inferred 边召回率 70%，是探针覆盖率的直接体现，而非缝合算法本身的有效性限制。具体而言，52 条静态边来自 llama.cpp 源码中“所有可能执行的路径”，但实际推理请求仅走其中子集；当前仅在 8 个 llama.cpp 函数上设置 uprobe，这 8 个函数内部调用的子函数均未被探针覆盖。值得注意的是，10% 召回率对应的 3 条 confirmed 边恰好是“最有价值”的 3 条——全部位于跨语言边界上，证明核心问题已被解决。

23 秒采样为 PoC 切片。当前 23.83 秒采样时长是用于验证分析框架可行性的 Proof of Concept 切片，长尾路径在 23 秒内未被触发属于采样窗口的自然限制，生产部署需延长采样时间以捕获更多边缘场景。

缝合算法在极端场景下存在局限。对于超长调用链（>20 层）和高度动态的反射调用

场景，当前基于锚点匹配的缝合算法可能失效。Go 运行时使用 `reflect` 包实现的动态方法调用无法通过静态符号表预测调用目标；C++ 虚函数的动态分派使得间接调用求解不完整。

异常检测依赖人工定义规则。当前基于规则的异常检测依赖人工定义模式，对未知异常类型的泛化能力有限，缺乏对异常行为的自适应学习能力。

6.3 未来工作展望

GPU 推理路径覆盖。将引入 NVIDIA GPU 追踪方案，具体包括：利用 `nvidia-smi dmon` 和 `nvidia-container-toolkit` 采集 GPU 利用率和显存使用数据；通过 CUDA Profiling Tools Interface (CUPTI) 追踪 CUDA API 调用（如 `cuMemcpy`、`cuMemAlloc`），建立从 `Python torch.nn.Module.forward` 到 CUDA kernel 的完整调用链；实现容器化推理环境下的 GPU 指标采集，将容器 GPU 指标与主机侧 eBPF 追踪数据进行关联分析。

提升探针覆盖率与缝合召回率。通过增加 `uprobe` 目标函数数量和延长采样时间，提升 `confirmed` 边的召回率；同时将采样窗口从 PoC 切片扩展为生产级基准数据集，捕获长尾调用路径。

极端场景缝合优化。引入 Go 运行时追踪获取实际执行的反射方法名并补充到动态轨迹中；利用 RTTI 信息建立类层次结构并结合 CFG 分析求解 C++ 虚函数的可能调用目标集合；长期可引入基于深度学习的调用链预测方法。

智能化异常检测。引入机器学习提升检测智能化水平，包括基于自编码器的异常检测、基于 LSTM 的时序异常检测以及构建正常行为基线，减少对人工定义规则的依赖。

6.4 本章小结

本文围绕开源 AI 推理引擎的跨语言调用链追踪问题，提出了基于 eBPF 的 HybriChain 方法，从跨语言符号映射、动态追踪采集、调用链缝合三个维度系统性地解决了混合编程架构下的调用链断裂难题。实验结果表明，HybriChain 能够在几乎不影响推理引擎性能的前提下，完成跨语言调用链的精确追踪，并发现推理过程中的可用性脆弱点。未来工作将在 GPU 推理路径覆盖、探针覆盖率提升和智能化异常检测三个方向继续深化研究。

参考文献

- [1] Georgi Gerganov. llama.cpp: Large language models in pure C/C++, 2023.
- [2] Georgi Gerganov. ggml: Tensor library for machine learning, 2023.
- [3] Xiaoyu Ma and David Patterson. Challenges and research directions for large language model inference hardware. *IEEE Computer*, 59(5):55–64, 2026.
- [4] Aditi M. Jain and Ayush Jain. Scaling LLM inference architectures: A performance analysis for chatbot applications. In *Proceedings of the 2025 6th International Conference on Artificial Intelligence, Robotics and Control (AIRC)*, pages 8–16, 2025.
- [5] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP '23)*, pages 611–626, 2023.
- [6] PyTorch Team. TorchServe: A flexible and easy to use tool for serving deep learning models, 2024.
- [7] Sagi Tzadik and Valentin Lobstein. Problama: Ollama remote code execution vulnerability (CVE-2024-37032), 2024.
- [8] Jonathan Corbet and Alexei Starovoitov. BPF: the universal in-kernel virtual machine. *LWN.net*, 2014.
- [9] Brendan Gregg. *BPF Performance Tools: Linux System and Application Observability*. Addison-Wesley Professional, 2020.
- [10] David Calavera and Lorenzo Fontana. *Linux Observability with BPF*. O'Reilly Media, 2019.
- [11] Andrii Nakryiko. BPF ring buffer, 2020.
- [12] Cilium Authors. cilium/ebpf: eBPF library for Go, 2024.
- [13] Yinan Hu. eBPF-based Observability for Serverless Wasm Workloads. Master's thesis, Aalto University, 2025.
- [14] Muntaka Ibnath, Mohammadreza Rezvani, and Daniel Wong. eBeeMetrics: An eBPF-based Library Framework for Feedback-free Observability of QoS Metrics. In *Proceedings of the 2026 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2026.
- [15] Bohua Zou, Debayan Roy, Dhimankumar Yogesh Airao, Weihao Xu, Binqi Sun, Yutao Liu, and Haibo Chen. ProfInfer: An eBPF-based Fine-Grained LLM Inference Profiler. In *Proceedings of the 9th Conference on Machine Learning and Systems (MLSys)*, 2026.
- [16] ControlPlane. eBPF Security Threat Model. Technical report, eBPF Foundation, 2024.
- [17] LinuxSecurity. Auditd vs eBPF: Modern Approaches to Linux System Monitoring, 2026.
- [18] Groundcover Team. What is eBPF? common use cases and best practices, 2024.

- [19] The Go Authors. cgo command – cmd/cgo, 2024.
- [20] Jinbao Chen, Boyao Ding, Yu prestige Zhang, Qingwei Li, and Fugen Tang. An empirical study of CGO usage in Go projects: Distribution, purposes, patterns and critical issues. *Journal of Systems and Software*, 231:112601, 2026.
- [21] Mingzhe Hu, Qi Zhao, Yu Zhang, and Yan Xiong. Cross-language call graph construction supporting different host languages. In *Proceedings of the 2023 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, pages 155–166, 2023.
- [22] Wenlong Zheng and Baojian Hua. Effective Call Graph Construction for Multilingual Programs (POLY-CALL). In *Proceedings of the 2023 IEEE European Symposium on Security and Privacy (EuroS&P)*, 2023.
- [23] Rose Sunil. LLMs for Call Graph Construction: Benchmarking and Evaluation Across Languages. Master’s thesis, Paderborn University, 2026.
- [24] Achim D. Brucker and Michael Herzberg. On the static analysis of hybrid mobile apps. In *International Symposium on Engineering Secure Software and Systems (ESSoS)*, pages 72–88, 2016.
- [25] Raoul Scholtes, Soheil Khodayari, Cristian-Alexandru Staicu, and Giancarlo Pellegrino. CHARON: Polyglot code analysis for detecting vulnerabilities in scripting languages native extensions. In *Proceedings of the 2025 IEEE European Symposium on Security and Privacy (EuroS&P)*, 2025.
- [26] Xiao Cheng, Haoyu Wang, Jiayi Hua, Guoai Xu, and Yulei Sui. Deepwukong: Statically detecting software vulnerabilities using deep graph neural networks. *ACM Transactions on Software Engineering and Methodology (TOSEM)*, 30(3):1–33, 2021.
- [27] Derek Bruening, Thomas Garnett, and Saman Amarasinghe. DynamoRIO: A framework for implementing dynamic analyses and optimizations. In *Proceedings of the International Symposium on Code Generation and Optimization (CGO)*, pages 265–275, 2003.
- [28] Chris Lattner and Vikram Adve. LLVM: A compilation framework for lifelong program analysis & transformation. In *Proceedings of the International Symposium on Code Generation and Optimization (CGO)*, pages 75–86, 2004.
- [29] Nicholas Nethercote and Julian Seward. Valgrind: A framework for heavyweight dynamic binary instrumentation. *Proceedings of the 28th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 42(6):89–100, 2007.
- [30] Nicholas Nethercote and Julian Seward. How to shadow every byte of memory used by a program. In *Proceedings of the 3rd ACM SIGPLAN/SIGOPS International Conference on Virtual Execution Environments (VEE)*, pages 65–74, 2007.

声 明

本人声明所呈交的学位论文是本人在导师指导下进行的研究工作及取得的研究成果。据我所知，除了文中特别加以标注和致谢的地方外，论文中不包含其他人已经发表或撰写过的研究成果，也不包含为获得四川大学或其他教育机构的学位或证书而使用过的材料。与我一同工作的同志对本研究所做的任何贡献均已在论文中作了明确说明并表示谢意。

本学位论文成果是在四川大学学习期间，在指导教师指导下取得的研究成果。论文成果归四川大学所有，特此声明。

作者签名：

导师签名：

年 月 日

学位论文使用授权书

本学位论文作者完全了解四川大学有关保留、使用学位论文的规定，同意学校保留并向国家有关部门或相关机构送交论文的原件、复印件和电子版，允许论文被查阅和借阅。本人授权四川大学将本学位论文的全部或部分内容编入有关数据库进行信息技术服务，可以采用影印、缩印或扫描等复制手段保存、汇编学位论文，并用于学术活动。

(涉密学位论文在解密后适用于本授权书)

作者签名：

导师签名：

年 月 日

四川大学本科毕业论文 AI 工具使用声明

本人声明，在撰写题目为《基于 eBPF 的开源 AI 推理引擎的跨语言调用链分析》的本科毕业论文过程中，对 AI 工具的使用情况如下：

一、使用的 AI 工具及使用目的

序号	AI 工具	使用目的
1	Cursor AI 编程助手	辅助论文文本润色与病句检查、LaTeX 排版问题排查与格式调整、公开资料检索思路整理、图表说明文字修改

二、使用程度说明

序号	章节	AI 工具	使用方式	处理方式
1	第 1 章绪论、第 2 章技术综述	Cursor AI 编程助手	输入特定段落请其提出语言优化建议，获取多种表述方案	本人对 AI 提供的内容进行筛选、修改、重新组织语言，结合研究内容进行调整，确保学术表述准确
2	第 3 章系统设计、第 4 章实现方案	Cursor AI 编程助手	辅助梳理系统架构描述逻辑、代码注释与流程图说明文字	本人对建议内容进行核实后采纳，确保技术描述与实际实现一致
3	第 5 章实验评估	Cursor AI 编程助手	辅助润色实验结果描述语句	本人结合实际实验数据对表述进行核实与修改，确保结论描述与数据一致

在论文的第 1 章绪论、第 2 章技术综述中使用了上述 AI 工具。使用方式为输入特定段落请其提出语言优化建议，获取多种表述方案。本人对 AI 工具提供的内容进行了筛选、修改、重新组织语言，结合自身研究内容进行调整，以确保论文内容符合学术要求及本人的研究思路。

三、原创性与责任声明

本人确认，尽管使用了上述 AI 工具，但论文整体的研究思路、核心观点、论证过程及最终结论均为本人独立思考与研究的成果。论文中引用 AI 工具生成内容之处，均已按照学校规定的学术规范进行了明确标注。

若论文因 AI 工具使用不当或未按要求标注等原因出现学术不端问题，本人愿意承担全部责任。

致 谢

首先，衷心感谢指导教师在选题确定、研究推进、论文修改与格式审阅过程中给予的耐心指导和帮助。导师严谨的治学态度与细致的工作方式，为论文的顺利完成提供了重要支持。

其次，感谢学院各位任课教师在本科阶段的培养。课程学习与实验训练为毕业论文的完成奠定了基础，也帮助我逐步建立起较完整的专业知识结构。

同时，感谢在论文撰写和排版整理过程中给予帮助的同学与朋友。大家在资料收集、内容讨论、格式调整与问题排查等方面提供了许多宝贵建议。

最后，感谢家人在整个本科阶段给予的理解、支持与鼓励。正是有了他们的陪伴与支持，我才能更加从容地完成学业和论文写作。

谨以此文，向所有关心、帮助和支持过我的老师、同学、朋友及家人致以诚挚的谢意。